

すばる式ローマ字 モーラ強調装飾ガイドライン 改訂版 Ver 2.3-r1

1. 章: このガイドラインの位置づけ

1.1 目的

本ガイドラインは、**すばる式ローマ字モーラ強調装飾 (SuBaRu Ro-Ma marks on mora)**の「標準モード」における表示仕様を定めることを目的とする。特に、トークン化・モーラ構築・表示分類から構成されるモーラパイプライン処理と、単語種別、変換処理の挙動を、創作用途と実装用途の双方から参照できる形で明文化する。

本書で定義する標準モードは、代表語テスト表 および V 系回帰テスト表に記載された入出力例と整合するよう設計されており、今後の拡張モード（読みやすさ重視モード・異質感重視モード等）の基準点としても機能する。

1.2 適用範囲

本ガイドラインが対象とするのは、次の条件を満たす環境および処理である。

- すばる式ローマ字モーラ強調装飾器は、トークン化 → モーラ構築 → 表示分類 の三段構成を採用していること。
- 単語種別により、入力語が
 - 日本語実用ローマ字
 - ラテン由来の言語
 - 判別不明 のいずれかに分類され、日本語実用ローマ字と判定された語のみが モーラ強調装飾の対象となること。
- 回帰テストプランに示された代表語セット（Q 基本セット、長音・eo/ae セット、ny 代表セット など）の入出力が、本書の仕様と一致していること。

本書は、すばる式ローマ字モーラ強調装飾の **表示仕様** を記述するものであり、ローマ字そのものの正書法を再定義するものではない。

標準的なローマ字表記（日本式・訓令式・ヘボン式・便宜表記など）の上に、「モーラ境界を強調する視覚的スタイル」を重ねるための仕様として位置づける。

1.3 標準モードの設計思想（拍優先・ノイズ許容）

本ガイドラインでいう「標準モード」とは、日本語実用ローマ字を対象に モーラパイプライン処理（トークン化 → モーラ構築 → 表示分類）を適用し、基本表示のみを出力する動作モードを指す。

表示例外や UI 固有の演出は、標準モードの結果に対する後段処理として扱う。

標準モードは、「日本語の拍（モーラ）が視覚的に一目で分かること」を最優先とし、意図的に一定の視覚的ノイズや、英語などとの混在文による“浮き上がり”効果で可視化する設計とする。

具体的には、- Q 系モーラの強調（GakKoU, TopPu）、- 長音・二重母音の顕在化（ToUKyoU, SeIKoU, KeIEI, OSaETe など）、- および Sin'Ya, Kin'YoUBi のような「n 条件や V 位置別既定表示」によるモチーフ語の崩しを、「標準モードにおける基本挙動」として採用する。

この結果、たとえば Keiei no seion in English のような文では、日本語実用ローマ字部分（KeIEI No SeIOu）だけが強く装飾され、英語部分（in English）とのコントラストが大きくなるが、敢えて狙った演出であり、誤りではない。より控えめな装飾や読みやすさを優先するバリエーションは、本書で定める標準モードを基準として、別途「派生モード」として設計するものとする。

1.4 ローマ字入力と日本語語の括弧補足方針

本書では、ローマ字入力そのものの見え方を観察対象とすることを基本とする。そのため、すべての想定入力文字列に対して一律に日本語語を併記するのではなく、必要な場合に限って、ローマ字入力の直後に括弧付きで想定する日本語語または想定読みを補足する。

括弧補足を付すのは、次の条件を満たす場合に限る。

- 当該語が、代表語セット（5.4, 7.1～7.3）または 6 章の個別例外として、明示的に参照される入力であること。
- 当該語の読み、元の語、または想定モチーフが、ローマ字だけでは直感的に推測しづらいこと。
- あるいは、夜 / 曜日 / 拡張候補語のように、仕様上の題材や観測意図を明示しておく必要があること。

たとえば、seikou に対して（成功）、keiei に対して（経営）、seion に対して（清音）、sieawo に対して（想定読み：シェアを）のような補足を付すことは、この方針に含まれる。

一方で、Q 基本セットのように、主眼が促音結合や表示規則そのものの確認にあり、元の日本語語を併記しなくても仕様理解に支障が小さいものについては、括弧補足を行わない。また、英語混在文の文脈例、ny 監視セット、純粋な英単語などについても、仕様上の見え方を優先し、原則として括弧補足の対象外とする。

本方針の目的は、説明のための補足情報を必要最小限にとどめつつ、ローマ字装飾そのものの視覚的観察を妨げないことである。したがって、括弧補足は辞書的注釈としてではなく、代表語の解釈補助および仕様意図の共有のために限定して用いる。

2. 章: 処理フロー概要（標準モード）

2.1 全体フロー概要

本節では、本ガイドラインで対象とする処理フロー全体の流れを示す。すばる式ローマ字モーラ強調装飾器は、入力された文章を単語単位に分解し、「単語種別の判定 → 日本語実用ローマ字だけをモーラ構造に変換 → モーラごとの表示スタイルを決定」という手順で、最終的な表示文字列を生成する。

処理の大まかな段階は、次の通りである。

1. **単語分割** 入力された文章を、空白や記号などを手がかりに「単語」へと切り分ける。

2. **単語種別の判定** 各単語について、「日本語実用ローマ字」「ラテン由来の言語」「判別不明」のいずれかに分類する。このとき、「no」など一部の語は**前後の文脈を参照して**、日本語の助詞とみなすかどうかを判定する。
3. **日本語実用ローマ字のモーラ化** 単語種別で「日本語実用ローマ字」と判定された単語についてのみ、まず子音と母音のまとまりに分解し（トークン化）、促音などを考慮して「1 モーラ単位」の列に組み立て直す（モーラ構築）。
4. **モーラごとの表示決定** モーラ列をもとに、子音と母音をどのような大文字・小文字の組み合わせで表記するかを決定する（表示分類）。この段階で、Q 系モーラ、長音、ei / eo / ae などの特殊パターン、および「n' 条件や V 位置別既定表示」による崩しが適用される。
5. **変換結果と三段ログの生成** 各単語ごとに「トークン列」「モーラ列」「最終表示」の三段ログをまとめ、それらを連結した全文の変換結果とともに出力する。三段ログは、回帰テストや仕様確認のための観察窓として利用する。

2.2 単語単位の処理と単語種別

単語単位の処理の入口となるのが「単語種別」である。ここでは、単語ごとに次の 3 区分のいずれかを与える。

- **日本語実用ローマ字** - 日本式・訓令式・ヘボン式・便宜表記などを含む、日本語を表記するためのローマ字。 - 例: `gakkou`, `seikou`, `Keiei`, `SinYa`, `KinYoUBi` など。
- **ラテン由来の言語** - 英語など、日本語ローマ字ではないラテン文字の単語。 - 例: `Japan`, `English`, `time`, `in`, `to`, `no`（英語としての `no`）など。
- **判別不明** - ラテン文字と非ラテン文字が混在しているもの、あるいはローマ字とも英単語とも判断しにくい文字列。 - 例: `abc東京`, `にほんgo` など。

この分類は、まず単語単体の形から判定し、その後で「no」のような曖昧な語だけ **左側（前の語）の文脈**を見て再判定する、という二段構えで行う。

単語種別で「日本語実用ローマ字」と判断された単語だけが、以降のモーラ構造への変換とモーラ強調装飾の対象となる。

単語種別の詳細な定義と判定基準は、3章「単語種別の仕様」で別途定める。

2.3 モーラパイプライン処理（日本語実用ローマ字のみ対象）

単語種別で「日本語実用ローマ字」と判定された単語は、次の二段階処理を経てモーラ単位の表示に変換される。

1. トークン化 → モーラ構築

- まず、子音＋母音、子音＋拗音＋母音、母音単独、撥音、促音といった単位に分解する。
- 次に、促音の直後に続く音節を 1 モーラにまとめるなどして、「1 モーラ = 1 要素」の列をつくる。
（例: `gakkou` → 「ga / k / ko / u」 → 「ga / kko / u」）

1. モーラごとの表示分類

- 各モーラについて、「子音だけ大文字」「子音 2 文字のうち片方だけ大文字」「母音を大文字にする」などのルールを適用し、目に見えるローマ字の形に変換する。
- この段階で、Q 系、長音、ei / eo / ae といったパターンや、SinYa, KinYoUBi のような「n' 条件や V 位置別既定表示」が反映される。

このモーラパイプライン処理は、標準モードにおけるもっとも特徴的な部分であり、代表語テスト表 (GakKoU, ToUKyoU, SeIKoU, KeIEI, OSaETe, SinYa, KinYoUBi など) の挙動がここで決まる。

モーラ構築の詳細な規則は5章で、V モーラに関する代表語と既定表示は7章および8章で、それぞれ具体的に定義する。

2.4 三段ログの構造

標準モードの変換処理は、「変換後の文章」とあわせて、各表記単位ごとに三段ログ（トークン列・モーラ列・最終表示）を記録する。

あわせて、単語本体と約物の区別、および単語種別などのメタ情報も 同じレコードとして保持することで、仕様確認と回帰テストの両方に利用できるようにする。

2.4.1 三段ログのフィールド構成

三段ログの 1 レコードは、概ね次のフィールド群から構成される。

- **toks-id**
 - 三段ログにおける 1 単語 (1 **core**) 単位の一意的識別子。
 - 入力行 × 単語順で採番し、実装ログやテスト結果との突合せキーとして用いる。
- **raw**
 - 入力文字列そのもの。前後の約物を含めた形で保持する。
 - 例: Sumutokoro,
- **core / core-type**
 - **core** は、**raw** から先頭・末尾の約物を取り除いた単語本体。
 - **core-type** は単語種別の判定結果であり、3 章の仕様と対応する。
 - **JP_ROMAJI**: 日本語実用ローマ字。モーラパイプライン処理の対象。
 - **LATIN_WORD**: ラテン由来の語 (英語など)。モーラ処理の対象外。
 - **UNKNOWN**: いずれにも分類しにくい文字列。モーラ処理の対象外。
- **core-toks** (core tokens)
 - **core** に対してトークン化を行った結果のトークン列。
 - 子音+母音、拗音、撥音、促音などの最小単位を **CV** / **CyV** / **V** / **N** / **Q** のラベル付きで記録する。
- **core-moras**
 - **core-toks** を仮名塊再解釈規則にしたがって再束ねした結果のモーラ列。
 - 「1 モーラ = 1 要素」となる列であり、V モーラ例外や Q 系のまとまり方を 確認するための中間結果となる。

- **core-out**
 - **core-moras** に対して表示分類（5 章）を適用した結果得られる、すばる式ローマ字による単語本体の最終表示。

読者やプレイヤーが目にする文字列のうち、約物を除いた部分に対応する。
- **punct / punct-type**
 - **punct** は、**core** の前後に付く約物（句読点・括弧・引用符など）を 1 本の文字列として保持するフィールドである。
 - 前処理では、**raw** から先頭側・末尾側の約物集合（`, . ! ? : ; ( ) [ ] { } " ' / -` など）を取り除き、取り除かれた部分を先頭約物列と末尾約物列として蓄積する。

これら先頭→末尾の順に連結したものを **punct** とする。

 - **punct-type** は **punct** の付着位置を示す識別子であり、次の 3 値をとる。
 - **LEFT_PUNCT**: **core** の先頭側にのみ約物が付いている場合。
 - **RIGHT_PUNCT**: **core** の末尾側にのみ約物が付いている場合。
 - **BOTH_PUNCT**: **core** の先頭・末尾の両側に約物が付いている場合。
 - **core** の内部に含まれるアポストロフィ `'` は、**Sin'Ya** のようなローマ字表記の一部として扱い、**punct** には移動しない。
- **output**
 - 実際に画面やテキストとして出力する最終表示。
 - **core-type** が **JP_ROMAJI** の場合は、基本的に **core-out + punct** となる。
 - **core-type** が **LATIN_WORD** や **UNKNOWN** の場合は、**core-toks / core-moras / core-out** は空とし、**output = raw** とする。
- **rules-applied**（任意）
 - **core-moras** から **core-out** を得る際に適用された表示規則の一覧。
 - 例: **5.7.2 + N→'n** のように、5 章の節番号や N モーラ変換を列挙する。
 - 仕様検証・デバッグ時に「どの V 例外や N モーラ規則が関与したか」を追跡するための補助フィールドであり、三段ログの必須 3 段（tokens / moras / result）とは独立している。

core-type が **JP_ROMAJI** の場合のみ、**core** に対して モーラパイプライン処理（トークン化 → モーラ構築 → 表示分類）を適用し、**core-toks / core-moras / core-out** を生成する。

それ以外の単語種別（**LATIN_WORD / UNKNOWN**）については、モーラ処理は行わず、**output** は **raw** と同一とすることを基本とする。

この構造により、三段ログは

- 入力の形（raw / core）
- モーラ構造（core-toks / core-moras）
- 表示結果（core-out / output）
- 約物の有無と位置（punct / punct-type）
- 適用された表示規則（rules-applied）

を一貫したフォーマットで記録し、仕様確認と回帰テストに同じログを再利用できるように設計されている。

2.4.2 約物付き入力の例

約物付き入力では、**raw** に約物を含めて保持しつつ、**core** / **punct** / **punct-type** を分けて記録することで、モーラ処理と最終表示を安定して観察できるようにする。

例 1: 文末ピリオド付きの挨拶

単語 **ohayou** にピリオドが付いた **Ohayou.** の例。

```
[Ohayou.]
toks-id   : T001
raw       : Ohayou.
core      : Ohayou
core-type : JP_ROMAJI
core-toks : CV:o | CV:ha | CV:yo | V:u
core-moras: CV:o | CV:ha | CV:yo | V:u
core-out  : OHaYoU
punct     : .
punct-type: RIGHT_PUNCT
rules-applied: (例) 5.2.x / 5.6.x など
output    : OHaYoU.
```

ここでは、**core** が日本語実用ローマ字としてモーラ処理の対象となり、末尾のピリオド **.** は **punct** として分離される。**punct-type** は、約物が語末のみに付いているため **RIGHT_PUNCT** となる。---

例 2: カンマ付きの感謝表現

arigatou にカンマが付いた **Arigatou,** の例。

```
[Arigatou,]
toks-id   : T002
raw       : Arigatou,
core      : Arigatou
core-type : JP_ROMAJI
core-toks : V:a | CV:ri | CV:ga | CV:to | V:u
core-moras: V:a | CV:ri | CV:ga | CV:to | V:u
core-out  : ARiGaToU
punct     : ,
punct-type: RIGHT_PUNCT
rules-applied: (例) 5.2.x / 5.6.x など
output    : ARiGaToU,
```

この例も、**core** 部分だけがモーラ処理の対象となり、文中の区切りとしてのカンマは **punct** に集約される。カンマが語末のみに現れるため、**punct-type** は **RIGHT_PUNCT** となる。

**例 3: 呼びかけに続く英語フレーズ

日常で頻出する呼びかけ+英語の混在フレーズとして、**Sumimasen, do you speak English?** を例として挙げる。

```
[Sumimasen, do you speak English?]
```

(1 語目)

```
toks-id    : T101
raw        : Sumimasen,
core       : Sumimasen
core-type  : JP_ROMAJI
core-toks  : CV:su | CV:mi | CV:ma | CV:se | N:n
core-moras: CV:su | CV:mi | CV:ma | CV:se | N:n
core-out   : SuMiMaSeN
punct      : ,
punct-type: RIGHT_PUNCT
rules-applied: (例) 5.2.x / 5.5
output     : SuMiMaSeN,
```

(2 語目)

```
toks-id    : T102
raw        : do
core       : do
core-type  : LATIN_WORD
core-toks  : (空)
core-moras: (空)
core-out   : (空)
punct      : (空)
punct-type: (空)
rules-applied: (空)
output     : do
```

(3 語目)

```
toks-id    : T103
raw        : you
core       : you
core-type  : LATIN_WORD
core-toks  : (空)
core-moras: (空)
core-out   : (空)
punct      : (空)
punct-type: (空)
rules-applied: (空)
output     : you
```

(4 語目)

```
toks-id    : T104
```

```
raw      : speak
core     : speak
core-type : LATIN_WORD
core-toks : (空)
core-moras : (空)
core-out  : (空)
punct    : (空)
punct-type : (空)
rules-applied : (空)
output   : speak

(5 語目)
toks-id   : T105
raw       : English?
core      : English
core-type : LATIN_WORD
core-toks : (空)
core-moras : (空)
core-out  : (空)
punct     : ?
punct-type : RIGHT_PUNCT
rules-applied : (空)
output    : English?
```

このフレーズでは、1 語目 *Sumimasen*, のみが `core-type = JP_ROMAJI` として モーラパイプライン処理の対象となり、日本語ローマ字として装飾される。

一方、*do / you / speak / English?* は `core-type = LATIN_WORD` として モーラ処理の対象外であり、`output` は基本的に `raw` と同一となる。

文頭と文末の約物（カンマと疑問符）は、それぞれ `punct` として分離され、`punct-type = RIGHT_PUNCT` で記録される。

これらの例のように、`raw / core-* / punct / punct-type / output` をあわせて記録しておくことで、入力の形、モーラ構造、表示結果、約物の有無と付着位置を、同一フォーマットの中で比較・確認できるようにする。

3. 章: 単語種別の仕様

3.1 単語種別の役割

本章では、「単語種別」の判定ルールを定める。単語種別は、入力された各単語を次の 3 区分のいずれかに分類し、「どの語にモーラ強調装飾を適用するか」を決定するための基準である。

- 日本語実用ローマ字
- ラテン由来の言語
- 判別不明

標準モードでは、「日本語実用ローマ字」と判定された単語にのみ、モーラ構造への変換とモーラごとの表示決定を行う。それ以外の単語は、そのまま（あるいは最小限の整形のみで）出力し、モーラ強調装飾の対

象にはしない。

3.2 日本語実用ローマ字

日本語実用ローマ字とは、日本語の語をローマ字で表したものであり、次のような系統を含む。

- 日本式ローマ字
- 訓令式ローマ字
- ヘボン式ローマ字（外務省ヘボン式など）
- 便宜的な実務表記（駅名標などで見られる混在表記を含む）

本ガイドラインにおいて「日本語実用ローマ字」とは、日本語の語をローマ字で表した文字列のうち、訓令式・日本式・ヘボン式・実務的な混在表記などを含む、実用的な綴り全般を指す。

厳密な方式（訓令式かヘボン式か）を細かく区別するのではなく、「日本語をローマ字で表すための文字列であり、かつローマ字として大きな破綻がないもの」を広く日本語実用ローマ字として扱う。

代表例として、次のような語が挙げられる。

- gakkou, toppu, kitte, kitteiru
- suu, oi, tou, toukyou, yasai
- seisyohou, seikou, keiei, seion
- siteori, osaete
- sinya, kinyoubi
- nyanko, nyuugaku, nyoro, ginyaku など。

これらの語は、単語種別の判定結果として「日本語実用ローマ字」となり、以降のモーラパイプライン処理の対象となる。

この単語種別は、標準モードでモーラパイプライン処理（トークン化 → モーラ構築 → 表示分類）を適用するかどうかを決める入口である。

日本語実用ローマ字と判定された語に対してのみ、N / Q / V モーラの再束ねや GakKoU, ToUKyoU, SeIKoU, KeIEI, OSaETe, SinYa, KinYoUBi といった特徴的な表示が生成される。

3.2.1 含める系統の例

日本語実用ローマ字には、おおむね次のような系統を含める。

- 日本式ローマ字
- 訓令式ローマ字
- ヘボン式ローマ字（外務省ヘボン式などを含む）
- 駅名標や看板などに見られる、方式が多少混在した実務表記

方式ごとの差異（例: shi / si, chi / ti など）は、モーラ構築段階で cv / Cyv へ正規化されることを前提とし、単語種別判定の時点では「日本語かどうか」の判定を優先する。

3.2.2 代表例

日本語実用ローマ字の代表例として、たとえば次のような語が挙げられる。

- Q 系・長音・基本例
 - gakkou, toppu, kitte, kitteiru
 - suu, oi, tou, toukyou, yasai
- V モーラ関連
 - seisyohou, seikou, keiei, seion
 - siteori, osaete, sieawo
- N / n' 関連
 - SinYa, KinYoUBi (表示上は Sin'Ya, Kin'YoUBi)
- ny 系監視用語
 - nyanko, nyuugaku, nyoro, ginyaku など。

これらの語は、単語種別判定の結果として「日本語実用ローマ字」と分類され、以降のモーラパイプライン処理の対象となる。

3.2.3 単語種別との関係

単語種別の判定は、まず単語単体の形から「日本語実用ローマ字」かどうかを推定し、no のような曖昧な語だけは左側の文脈を参照して「日本語の助詞としての no」か「英語の no」かを再判定する二段構えで行う。

この判定で「日本語実用ローマ字」となった語だけが、N / Q / V の再束ね・独立母音化・V-eo/ae/ea 特例など、すばる式ローマ字特有のモーラ強調装飾の対象となる。

3.3 ラテン由来の言語

ラテン由来の言語とは、英語など、日本語ローマ字として扱わず、そのままラテン文字の単語として扱うものを指す。

代表例として、次のような語がある。

- 英語の単語
 - Japan, Tokyo, English, time, nylon, case など。
- 英語の機能語
 - the, of, and, in, to, no など。

これらは、たとえ「ローマ字としても読めそうな綴り」であっても、本仕様では「日本語実用ローマ字」とは見なさず、「ラテン由来の言語」としてモーラ強調装飾の対象外とする。

英語混在文において、ラテン由来の言語はそのままの綴りで残り、日本語ローマ字部分だけが強く装飾されることで、前章で述べた「浮き上がり」効果が生じる。

3.4 判別不明

判別不明とは、次のような理由から「日本語実用ローマ字」とも「ラテン由来の言語」とも決めがたい単語を指す。

- ラテン文字と非ラテン文字（漢字・ひらがななど）が混在している場合
 - 例: **abc**東京, に**ほん**go など。
- 特殊記号や制御文字などが含まれ、どちらのカテゴリにも自然に収まらない場合

判別不明と分類された単語は、モーラ強調装飾の対象から外し、原則としてそのままの文字列を出力する。

これにより、仕様上想定していない文字列が入力された場合にも、モーラ処理系が **同じ入力に対して必ず同じ出力で返す** ようにする。

3.5 単語単体での判定（base 判定）

単語種別の判定は、まず「単語単体の形」に基づく base 判定から始める。base 判定では、次のような順序でカテゴリを決める。

1. 前処理

- 前後の空白を除去し、空であれば「判別不明」とする。
- 大文字・小文字の揺れは吸収し、判定には小文字化した形を用いる。

2. 固有名の判定

- 国名・都市名・言語名など、事前に定めた固有名リストに含まれるものは、
- 「ラテン由来の言語」として扱う。
- 例: **japan**, **tokyo**, **english** など。

3. 機能語の判定

- **the**, **of**, **and**, **in**, **to**, **no** など、一部の機能語は
- 常に「ラテン由来の言語」として扱う。
- **no** については、後述の文脈補正で「日本語の助詞」と見なされる場合があるが、base 判定の段階では英語の一部として扱う。

4. ローマ字としての妥当性の確認

- 残りの単語について、日本語ローマ字として妥当な子音＋母音の組み合わせかどうかを確認する。

- 妥当な組み合わせのみから構成されていれば「日本語実用ローマ字」、そうでなければ「ラテン由来の言語」とみなす。

5. 非ラテン混在の確認

- ラテン文字以外（漢字・かな・記号など）が混在している場合は、「判別不明」とする。

この base 判定の結果は、「日本語実用ローマ字」「ラテン由来の言語」「判別不明」のいずれかであり、通常はこのまま単語種別の最終結果として用いられる。

ただし、「no」に限っては、次節の文脈補正によって種別が変更されることがある。

3.6 文脈補正（no のみ）

文中の「no」は、単語単体で見ると英語の機能語と日本語の助詞の両方になり得るため、base 判定ではいったん「ラテン由来の言語」として扱う。

そのうえで、前後の文脈を参照し、日本語の助詞とみなせる場合に限り、「日本語実用ローマ字」に昇格させる。

3.6.1 補正対象

- 補正の対象となるのは、綴りが **no** である単語のみとする。
- それ以外の曖昧語（**to**, **in** など）は、本仕様の文脈補正の対象外とする。

3.6.2 前後 2 語に基づく補正ルール

base 判定で「ラテン由来の言語」となった **no** について、前後最大 2 語までの単語種別を参照し、次の条件を満たす場合に **no** を日本語実用ローマ字（助詞）として扱う。

- 条件: **no** の前後 2 語のうち、少なくとも 1 語が日本語実用ローマ字であること

no の 2 語前、1 語前、1 語後、2 語後のいずれかに、日本語実用ローマ字が存在する場合、**no** は日本語の「の」である可能性が高いとみなし、日本語実用ローマ字（助詞）として補正する。

例:

- **gakkou no seikou**
 - 前: **gakkou**（日本語実用ローマ字）
 - 後: **seikou**（日本語実用ローマ字）
 - → 近傍に日本語実用ローマ字が存在するため、**no** を助詞とみなし **No** に補正する。
出力: **GakKoU No SeIKoU**
- **Keiei no seion in English**
 - 前: **Keiei**（日本語実用ローマ字）
 - 後: **seion**（日本語実用ローマ字）、そのさらに後ろに英語

- → 近傍に日本語実用ローマ字が存在するため、**no** を助詞とみなし **No** に補正する。

出力: **KeIEI No SeIO n in English**

- **Japan no keiei**

- 前: **Japan** (ラテン由来の言語)
- 後: **keiei** (日本語実用ローマ字)
- → **no** の直後に日本語実用ローマ字が存在するため、助詞とみなし **No** に補正する。

出力: **Japan No KeIEI**

3.6.3 補正しないケース

no の 2 語前、1 語前、1 語後、2 語後のいずれにも 日本語実用ローマ字が存在しない場合、**no** は英語の一部とみなし、単語種別は「ラテン由来の言語」のままとし、表示も小文字の **no** のままとする。

例:

- **in no case**
 - 前: **in** (ラテン由来の言語)
 - 後: **case** (ラテン由来の言語)
 - 近傍 2 語の範囲に日本語実用ローマ字が存在しないため、
 - **no** は英語表現の一部として扱われる。

出力: **in no case**

3.7 単語種別テストケースへの参照

単語種別の判定が仕様どおりに動作しているかを確認するため、本書の巻末「テストケース一覧」には、単語単体の判定テーブル および文脈付きの判定テーブルを掲載する。

- 単語単体テーブルでは、**gakkou, SinYa, nyanko, ginyaku** などが「日本語実用ローマ字」。
- **japan, nylon, the, no** などが「ラテン由来の言語」。
- **abc東京, にほんgo** などが「判別不明」と判定されることを確認する。
- 文脈付きテーブルでは、**gakkou no seikou, Keiei no seion in English, Japan no keiei** などの例について、**no** の補正有無が、上記ルールどおりであることを確認する。

これらのテストケースは、単語種別ロジックを変更した際の 回帰チェックに必ず用いるものとする。

4. 章: モーラ構築仕様

4.1 本章の役割

本章では、トークン化された日本語実用ローマ字を、どのように「1 モーラ単位」の列へ組み立てるかを定める。

ここで扱うのはあくまで**構造の確定**であり、大文字・小文字の見え方や、長音・二重母音の強調のしかたは、次章以降の表示分類で扱う。

標準モードにおけるモーラ構築は、できる限り単純な規則を保つことを基本方針とする。

そのため、特殊な結合は促音系に限定し、原則として「1 トークン = 1 モーラ」の対応を維持する。

4.2 トークンとモーラの関係

モーラ構築の入力となるのは、すでにトークン化された単位列である。

この段階では、子音+母音、子音+拗音+母音、母音単独、撥音、促音などが、それぞれ個別のトークンとして並んでいることを前提とする。

標準モードでは、モーラ構築の基本原則を次のように定める。

- 原則として、1 つのトークンはそのまま 1 モーラとみなす。
- ただし、促音を表すトークンだけは、その直後の音節と結合して 1 モーラを構成する。

この設計により、モーラ構築の責務を最小限に保ちつつ、促音を含む語でも、日本語として自然な拍のまとまりを得ることができる。

4.3 基本規則

標準モードにおけるモーラ構築の規則は、次の 3 つである。

1. **促音の直後に通常の音節が続く場合** 促音トークンの直後に、子音+母音からなる音節があるときは、両者を結合して 1 モーラとする。

これは、 $Q + CV \rightarrow QCV$ に相当する規則である。

1. **促音の直後に拗音を含む音節が続く場合** 促音トークンの直後に、子音+拗音+母音からなる音節があるときも、両者を結合して 1 モーラとする。

これは、 $Q + CyV \rightarrow QCyV$ に相当する規則である。

1. **それ以外は結合しない**

上記以外の組み合わせについては、すべて「1 トークン = 1 モーラ」として扱う。

この 3 規則によって、標準モードでは「促音だけを特別扱いし、それ以外は過度にまとめない」という構造が保たれる。

長音、二重母音、ei / eo / ae などは、この段階では まだ 1 モーラごとに保持され、後続の表示分類で強調のしかたを決める。

4.4 促音系の構築例

促音を含む語では、トークン列とモーラ列のあいだに差が生じる。

以下は、標準モードにおける代表的な構築例である。

- **gakkou**
 - トークン列（概念）：**ga** / **k** / **ko** / **u**
 - モーラ列（概念）：**ga** / **kko** / **u**
 - 促音 **k** は直後の **ko** と結合し、**kko** で 1 モーラとなる。
- **toppu**
 - トークン列（概念）：**to** / **p** / **pu**
 - モーラ列（概念）：**to** / **ppu**
 - 促音 **p** は直後の **pu** と結合する。
- **kitte**
 - トークン列（概念）：**ki** / **t** / **te**
 - モーラ列（概念）：**ki** / **tte**
 - 促音 **t** は直後の **te** と結合する。
- **kitteiru**
 - トークン列（概念）：**ki** / **t** / **te** / **i** / **ru**
 - モーラ列（概念）：**ki** / **tte** / **i** / **ru**
 - 促音部分だけが結合し、その後ろの母音 **i** は独立したモーラのまま保持される。

これらの例から分かるように、モーラ構築の段階で変化するのは 促音周辺に限られ、それ以外の要素はできるだけ元のトークンの並びを保つ。

4.5 長音・二重母音・単独母音の扱い

長音、二重母音、ei / eo / ae などを含む語では、表示上は強い変化が現れる場合があるが、モーラ構築の段階ではそれらをまだ統合しないことが標準モードの原則である。

たとえば、次のような語でも、母音モーラは独立したまま保持される。

- **suu**
 - モーラ列（概念）：**su** / **u**
- **tou**
 - モーラ列（概念）：**to** / **u**
- **toukyou**
 - モーラ列（概念）：**to** / **u** / **kyo** / **u**
- **seikou**
 - モーラ列（概念）：**se** / **i** / **ko** / **u**

- **seion**
 - モーラ列（概念）：**se / i / o / n**
- **siteori**
 - モーラ列（概念）：**si / te / o / ri**
- **osaete**
 - モーラ列（概念）：**o / sa / e / te**

これらは後段で **ToUKyoU**, **SeIKoU**, **SeIOn**, **SiTeORi**, **OSaETe** のような 強調表示になるが、それは**表示分類の規則**によるものであり、モーラ構築そのものは単純な列の保持にとどまる。

4.6 モーラ構築の設計意図

標準モードでは、モーラ構築の段階に複雑な例外を持ち込まない。その理由は、構造の決定と見た目の演出を分離しておくことで、仕様全体の見通しをよくし、回帰テストもしやすくするためである。

もし長音や特定の二重母音までこの段階で結合してしまうと、- 構造の規則 - 見た目の強調規則 - 「n' 条件や V 位置別既定表示」の規則

が一つの層に混ざり、どこを変更したときに何が壊れたのかが分かりにくくなる。

そのため本仕様では、モーラ構築は「促音を自然な 1 拍としてまとめる」ことに集中し、それ以外の演出は表示分類側に委ねる。

4.7 回帰テストにおける確認対象

モーラ構築の変更を行った場合は、少なくとも Q 基本セットに対して、トークン列とモーラ列の対応が維持されていることを確認する。

- **gakkou** → **ga / k / ko / u** から **ga / kko / u** になること。
- **toppu** → **to / p / pu** から **to / ppu** になること。
- **kitte** → **ki / t / te** から **ki / tte** になること。
- **kitteiru** → **ki / t / te / i / ru** から **ki / tte / i / ru** になること。

あわせて、長音・二重母音を含む語については、不要なモーラ結合が起きていないことも確認する。

たとえば **toukyou** が **to / ukyo / u** のように 不自然にまとめられず、**to / u / kyo / u** のまま保持されていることは、標準モードの重要な前提である。

5. 章: 表示分類仕様（標準モード）

5.1 表示分類の役割と前提

本章では、モーラ構築によって得られたモーラ列を、最終的にどのようなローマ字表示へ変換するかを定める。

ここで扱うのは、各モーラ種別に対して、子音と母音を どのような大文字・小文字の組み合わせで視覚化するか、という表示上の規則である。

標準モードの表示分類は、「自然なローマ字綴りを保つこと」よりも、「どこに拍の区切れ目があるかを視覚的に分かりやすくすること」を優先する。

そのため、英語風の視点から見るとやや不自然な大文字配置が生じる場合があるが、それは拍の可視化を優先した結果であり、本仕様における意図的な特徴である。

また、本章で定めるのは、あくまで**基本表示規則**である。

長音、二重母音、ei / eo / ae、「n' 条件や V 位置別既定表示」などの 追加的な変化は、後続の節で別途定義する。

5.2 モーラ種別ごとの基本表示ルール

標準モードでは、モーラ構築によって得られた各モーラを、その種別に応じて次のように表示する。

5.2.1 子音+母音モーラ

子音+母音から成る通常のモーラは、**子音を大文字、母音を小文字**で表示する。これは標準モードにおける最も基本的な表示形であり、多くの語で骨格となる見え方である。

例:

- ga → Ga
- sa → Sa
- ki → Ki
- ru → Ru

この規則により、通常の拍は「立ち上がりだけが強調された形」に揃えられ、モーラ単位の区切りが視覚的に把握しやすくなる。

5.2.2 拗音を含むモーラ

子音+拗音+母音から成るモーラは、**先頭の子音のみを大文字**とし、残りは小文字で表示する。すなわち、標準モードでは拗音モーラ全体を 1 拍として維持しつつ、先頭だけを立てる形を採る。

例:

- kyo → Kyo
- syo → Syo
- nya → Nya
- nyo → Nyo

この規則は、「拗音を 1 モーラとして読む」という日本語側の感覚を保ちながら、見た目のノイズを過剰に増やしすぎないための基本形でもある。

5.2.3 促音を含む結合モーラ

促音とその直後の音節が結合してできたモーラは、**先頭の子音を小文字、次の子音を大文字、母音を小文字**で表示する。

これは標準モードにおける Q 系モーラの基本表示であり、詰まり感と拍の切れ目を同時に見せるための重要な規則である。

例:

- tte → tTe
- kko → kKo
- ppu → pPu

たとえば gakkou では ga / kko / u のうち kko が kKo となり、GakKoU の中央に強い視覚的のひっかかりが生まれる。

このひっかかりは、標準モードが持つ「読めるが少し異様」の感触を支える主要な要素である。

語末に撥音 N を伴う場合、Q + CV + N を 1 つの重い仮名塊として再束ねすることがある。このとき、促音と語末撥音を含む 1 モーラを後続側にまとめ、その塊の先頭子音を大文字化することで、促音起源の子音肥大と語末撥音を同時に視覚的に示すことができる。

例: nippon (にっぽん)

- モーラ列 (再解釈) : ni / ppon
- 表示: NipPon

ここでは、「にっぽん」の第 2 拍「ぽん」を ppon という 1 モーラ塊として扱い、P を立てることで「促音 + 語末撥音を含む重い拍」であることを視覚的に示している。

nippon → NipPon は、Q 系モーラの基本表示と CVn (撥音を直前塊へ従属させる再束ね) 規則から自然に導かれる標準表示として扱う。語末 n は独立化しない。語末 N を CVn として直前塊へ従属させる。

5.2.4 撥音モーラ

撥音モーラは、常に 'n (アポストロフィ + 小文字 n) の 2 文字で表示する。

例:

- n → 'n

このルールにより、撥音は他の na / ni / nu / ne / no などと視覚的に明確に区別され、「子音 + 母音の 1 拍」と混同されにくくなる。

また、語中の n が - 撥音モーラ ('n) - 次の母音と結びついた音節 (na / ni など) のどちらなのかが一目で分かるようになり、誤読や曖昧さの低減につながる。

5.2.5 単独母音モーラの基本形

単独母音モーラについては、まずデフォルト表示を定め、そのうえで次節以降の例外規則を上書きする。

標準モードにおける単独母音モーラの基本形は、**語中では小文字、語末では大文字**とする。

例:

- 語中の **a / i / u / e / o** → **a / i / u / e / o**
- 語末の **a / i / u / e / o** → **A / I / U / E / O**

この規則により、単独母音は通常時には控えめに扱われる一方、語末に現れたときだけ拍の終止として強く見える。

たとえば **tou** の末尾 **u** はデフォルト規則だけで **U** となり、**ToU** という出力になる。

ただし、**suu**, **toukyou**, **seikou**, **keiei**, **seion**, **siteori**, **osaete** などでは、単独母音モーラに対して追加の例外規則が適用され、語中でも大文字になる場合、逆に語末でも小文字のまま据え置かれる場合がある。

5.3 V モーラ例外の評価順

単独母音モーラは、標準モードにおいてもっとも変化の大きい要素である。そのため、5.2.5 で定めたデフォルト表示をそのまま適用するのではなく、まず例外規則を**上から順に評価**し、いずれにも該当しなかった場合にのみデフォルト表示を用いるものとする。

この評価順は、単に説明の都合で並べたものではなく、実際の表示結果を安定させるための優先順位でもある。より限定的な条件を先に評価し、より一般的な条件を後ろに置くことで、代表語テスト表の期待値が再現されるように設計する。

5.3.1 促音後の **i** を優先する規則

まず、促音を含む結合モーラの直後に現れる **i** で、さらにその後ろに通常の子音+母音モーラが続く場合は、その **i** を大文字 **I** として表示する。

この規則は、代表例 **kitteiru** のような語に対して適用される。想定モーラ列が **ki / tte / i / ru** のとき、**tte** の直後にある **i** がこの規則に該当し、出力は **KitTeIRu** となる。

この規則を最優先に置く理由は、促音を含む塊の直後に来る母音を強く立てることで、**KitTeIRu** のような語において、中央の拍のつながりと切れ目を同時に見せやすくするためである。

5.3.2 **e + i** 系を一般化する規則

次に、直前のモーラが子音+母音モーラであり、その母音が **e**、かつ現在の単独母音モーラが **i** である場合、その **i** を大文字 **I** として表示する。

この規則は、**seisyohou**, **seikou**, **keiei**, **seion** などの代表語を対象とする。たとえば **seikou** では **se / i / ko / u** の **i** が **I** となり、**SeIKoU** となる。同様に、**keiei** は **KeIEI**、**seion** は **SeIOn** となる。

この規則は、標準的なローマ字の見た目を大きく崩しすぎずに、**e** に続く独立母音 **i** を「別拍として立てる」ための中核ルールである。

5.3.3 **suu** における末尾 **u** の据え置き規則

次に、子音＋母音モーラ **su** の直後に単独母音 **u** が置かれ、かつその **u** が語末にある場合は、語末であってもその **u** を大文字にせず、小文字のまま保持する。

この規則は、代表例 **suu** に対して適用され、出力を **Suu** とするための特例である。

もしこの規則がなければ、5.2.5 のデフォルト規則により **SuU** あるいはそれに類する強い見え方になり、標準モードの代表語として想定しているバランスから外れてしまう。

これは、「語末だから必ず大文字にする」という単純規則に対して、**su + u** だけは例外的に滑らかさを優先するための調整である。

5.3.4 中間 **u** を立てる規則

次に、単独母音 **u** が語中にあり、その前が **to**、その後ろに拗音を含むモーラが続く場合、その **u** を大文字 **U** として表示する。

この規則は、代表例 **toukyou** において **to / u / kyo / u** の 2 モーラ目の **u** に適用される。その結果、語中の **u** は **U** として立ち上がり、出力は **ToUKyoU** となる。

一方、**tou** のように後続が拗音ではない場合には、この規則は適用されない。

その場合は語末大文字のデフォルト規則によって **ToU** となるため、**tou** と **toukyou** は似た綴りでありながら、語中と語末で異なる理由で区別される。

5.3.5 **te + o** を立てる規則

次に、単独母音 **o** の直前に **te** がある場合、その **o** を大文字 **O** として表示する。

この規則は、代表例 **siteori** に対して適用される。

想定モーラ列 **si / te / o / ri** において、**te** の直後に置かれた **o** が強調され、出力は **SiTeORi** となる。

この規則は、**eo** の並びを「単なる続きの母音」として流さず、**te** のあとに別拍の **o** が立ち上がる感覚を強く見せるためのものである。

5.3.6 **osaete** 型の語頭 **o** と中間 **e** を立てる規則

次に、語頭が単独母音 **o** で始まり、その後に **sa**、さらに単独母音 **e**、その後に通常モーラが続く場合は、語頭の **o** を大文字 **O**、**sa** の直後にある **e** を大文字 **E** として表示する。

この規則は、代表例 **osaete** を **OSaETe** とするためのものである。想定モーラ列 **o / sa / e / te** に対して、先頭の **o** と中間の **e** の両方が強調されることで、語全体に標準モード特有の「拍ごとの引っぱり」が生まれる。

この規則は、語頭母音と中間母音を同時に立てる少し強めの演出であり、**siteori** や **seikou** よりも一段強いノイズ感を許容する例外として位置づけられる。

5.3.7 **e + a** が連続する拡張形では、**e** と **a** を表示上の強調対象とする

- sieawo → SieAWo

この規則は代表例 sieawo を対象とする。

5.3.8 語末 N を CVn に束ねる規則

- nippon → NipPon

この規則の代表例として nippon を挙げる。

想定モーラ列 ni / ppo / N に対して、語末 N を独立モーラとせず直前モーラに束ねることで、NipPon として 2 拍目の子音群 pP を強く立てる強調表示する。

※ Ver 2.3 では、NipPon の仕様上の扱いを scope-flag: extension から scope-flag: current に昇格させ、正式仕様として採用した。

5.3.9 デフォルト規則へのフォールバック

上記のいずれの例外にも該当しない単独母音モーラについては、5.2.5 の基本規則に戻り、語中では小文字、語末では大文字として表示する。

このフォールバックにより、例外規則は必要最小限に限定され、未定義の語に対しても安定した表示が得られる。

代表例 tou の語末 u のように、特別な例外ではなく デフォルト表示によって成立する語もある。

5.3.10 評価順を変えてはならない理由

V モーラ例外は、複数の規則にまたがって見える語が存在するため、評価順を入れ替えると結果が変わる可能性がある。

そのため、本仕様では 5.3.1 から 5.3.7 までの順序自体を 表示分類の一部として扱い、実装時にもこの順番を維持することを求める。

とくに kitteiru, seikou, keiei, seion, suu, toukyou, siteori, osaete は、V モーラ規則の変更時に最優先で回帰確認すべき代表語である。

5.4 代表語セットで見る表示例

この節では、これまでに説明したモーラ構築と表示分類の規則が 実際の単語でどのような形になるかを、代表語セットごとに示す。

各セットは、必須回帰テスト表の「1. 代表語テスト表」と直接対応しており、ここで挙げる例はすべて標準モードでの出力を対象とする。

なお、本節の input 欄に示すのは日本語実用ローマ字（小文字）であり、撥音モーラは n（必要に応じて n'）で表記する。

5.4.1 代表語テーブル（日本語実用ローマ字入力）

この表では、入力を「日本語実用ローマ字（小文字）」として扱い、V 系モーラ（長母音・二重母音・V-eo/ae/ea 特例）の代表語を示す。

tokens / moras / result は、それぞれ三段ログにおける「トークン列」「モーラ列」「最終表示」に対応する。

input	tokens	moras
result	notes	
suu	`CV:su \   V:u`	`CV:su \   V:u`
Suu	長母音 uu、日本語語「すう」等の代表。	
oi	`V:o \   V:i`	`V:o \   V:i`
Oi	一般二重母音 oi、日本語語「おい」等の代表。	
tou	`CV:to \   V:u`	`CV:to \   V:u`
ToU	ou 語末・長音寄り既定値、日本語語「とう」等の代表。	
yasai	`CV:ya \   CV:sa \   V:i`	`CV:ya \   CV:sa \   V:i`
YaSaI	語末 ai 分割既定、日本語語「やさい」。	
gyoukai	`CyV:gyo \   V:u \   CV:ka \   V:i`	`CyV:gyo \   V:u \   CV:ka \   V:i`
GyoUKaI	ou 語中+ai 語末の分割寄り、日本語語「業界」。	
kaihatu	`CV:ka \   V:i \   CV:ha \   CV:tu`	`CV:ka \   V:i \   CV:ha \   CV:tu`
KaIHaTu	語中 ai 分割既定、日本語語「開発」。	
seikou	`CV:se \   V:i \   CV:ko \   V:u`	`CV:se \   V:i \   CV:ko \   V:u`
SeIKoU	ei 語中+ou 語末の代表、日本語語「成功」。	
keiei	`CV:ke \   V:i \   V:e \   V:i`	`CV:ke \   V:i \   V:e \   V:i`
KeIEI	V-ei 系分割・強調の代表、日本語語「経営」。	
seion	`CV:se \   V:i \   V:o \   N:n`	`CV:se \   V:i \   V:o \   N:n`

SeIO _n	V-io 系分割・語末撥音付き、日本語語「清音」。	
siteori	`CV:si \   CV:te \   V:o \   CV:ri`	`CV:si \   CV:te \   V:o \   CV:ri`
SiTeORi	V-eo 系独立母音化代表、日本語語「しており」。	
osaete	`V:o \   CV:sa \   V:e \   CV:te`	`V:o \   CV:sa \   V:e \   CV:te`
OSaETe	V-ae 系独立母音化代表、日本語語「おさえて」。	
sieawo	`CV:si \   V:e \   V:a \   CV:wo`	`CV:si \   V:e \   V:a \   CV:wo`
SieAWo	V-ea 特例、想定読み「シェアーを」。現行仕様。	

このテーブルは、Ver 2.2 系における V モーラ表示の基本セットであり、7 章の V 代表語一覧および V 系回帰テスト計画（Block V）の期待値定義と一対一で対応する。

5.5 N モーラ表示（概念 N と表示 'n'）

撥音モーラは、本ガイドラインにおいてもっとも単純な扱いを採る。内部表現（モーラ列）と最終表示の双方で特別な一般化規則を設けず、「常に同じ記号で現れる拍」として位置づける。

ただし、実装上はモーラ列と最終表示のあいだに 1 段の変換が存在する。モーラ列では抽象記号 N を用い、最終表示では 'n' を用いる。

この 2 層構造は、4 章のモーラ構築と 5 章の表示分類の責務分割を明確にするためのものであり、機能としては「N モーラ = 'n 表示」の固定マッピングを意味する。

5.5.1 モーラ列における N

モーラ列（内部表現）の段階では、撥音モーラは常に N という抽象記号で表す。

この N は、かな／ローマ字入力に依存しない「撥音 1 拍」の概念ラベルとして扱い、n と m の表記差や綴り揺れには引きずられない。

たとえば seion の想定モーラ列は se / i / o / N として記録される。ここでは、末尾の N が「撥音モーラである」という事実だけを保持し、どのように表示するかは後続の表示分類フェーズに委ねる。

5.5.2 最終表示における 'n

最終表示の段階では、撥音モーラは常に 'n（アポストロフィと小文字 n）の 2 文字で表す。この 'n 表記は、通常の n と視覚的に区別しつつ、拍としての存在感を一定に保つためのデザインである。

たとえば seion は、モーラ列 se / i / o / N から表示列 SeIO_n に変換される。ここで、末尾の N は 'n に対応しており、SeIO_n における O_n の n は必ず 'n の形で出力される。

5.5.3 変換規則としての「N → 'n」

表示分類フェーズでは、N モーラに対して次の固定変換を適用する。

- モーラ列に N が現れた場合、その位置の最終表示は 'n とする
- N モーラに対しては、大文字・小文字の例外規則は適用しない
- N モーラは、V モーラ例外の評価順（5.3）からも独立して扱う

これにより、V モーラの大文字化・小文字化がどれだけ複雑になっても、撥音の見え方は常に 'n として安定する。

seion のように V 例外と N モーラが同居する語であっても、N 側の仕様は「常に 'n」という最小単位の約束で済む。

5.5.4 代表例

N モーラ表示の仕様を明示的に確認すべき代表語は次の通りである。

- seion
- モーラ列（概念）：se / i / o / N
 - 最終表示：SeIOⁿ
 - V モーラ例外（e + i 系一般化）と N → 'n の両方が正しく機能しているかを確認する代表例である。
- nippon
- モーラ列（概念）：ni / ppo / N
 - 最終表示：NipPon
 - 語末 N を独立させず CVⁿ として直前モーラに束ねることで、子音群 `pP` の強調と N モーラ再束ね挙動を確認する代表例である。

これらの語は、V モーラ例外の変更にかかわらず、N モーラ表示が 'n のまま維持されているか、また語末 N を CVⁿ として束ねる仕様が崩れていないかを、回帰テストで確認するための監視対象でもある。

5.6 V モーラ表示の基本（デフォルトは語末のみ大文字）

単独母音モーラ（以下、V モーラ）は、標準モードにおいて 拍の境界をもっとも揺らしやすい要素である。そのため本仕様では、V モーラの表示を単なる装飾規則としてではなく、表示全体の設計思想に関わる基本ルールとして扱う。

標準モードでは、V モーラに対して「語中では小文字、語末では大文字」というデフォルト表示を採用する。この方針は、可読性よりも拍の見えやすさを優先しつつも、語中の母音を過剰に立てすぎないための折衷である。

5.6.1 基本方針

V モーラは、子音を伴わない独立した母音拍である。標準モードでは、この独立性を常に強く見せるのではなく、語の途中では周囲の CV / CyV モーラとの連続感を優先し、語末でのみ拍の終止を明確に見せる方針を採る。

したがって、例外規則が適用されない限り、語中の V モーラは小文字のまま流し、語末の V モーラだけを大文字で表示する。この「語中小文字／語末大文字」が、V モーラ表示のデフォルトである。

5.6.2 語中 V を小文字にする理由

語中の V モーラまで常に大文字で立ててしまうと、標準モード全体が過度にノイジーになり、拍の見えやすさよりも視認負荷のほうが強くなってしまふ。そのため、語中の V はまず小文字で流し、必要な場合だけ例外規則によって立てる構成にしている。

この考え方により、CV / CyV モーラが担う「拍の骨格」と、V モーラが担う「骨格のあいだの伸び・切れ目・余韻」の役割分担が明確になる。V モーラは重要だが、常に前面へ出すのではなく、必要な箇所だけ強調する対象として扱う。

5.6.3 語末 V を大文字にする理由

語末の V モーラは、1 語の終わりを支える拍として働くため、標準モードでは原則として大文字で表示する。これにより、最後の母音が単なる綴りの余りではなく、独立した 1 拍として終止していることを視覚的に示せる。

たとえば yasai は、想定モーラ列を ya / sa / i とみなすことができ、末尾の i は語末 V モーラである。このためデフォルト表示では YaSaI となり、末尾の拍だけが一段強く立ち上がる。

5.6.4 CV / CyV との役割分担

子音＋母音モーラ (CV / CyV) は、標準モードにおいて 子音大文字＋母音小文字を基本とする。これに対して V モーラは、子音を持たない拍として、語中では控えめに流され、語末では終止を示す役割を担う。

この構造により、CV / CyV は拍の骨格、V は骨格の切れ目や余韻を調整する要素として機能する。V モーラの例外規則を後段で多数定義する場合でも、まずこのデフォルト構造が基準点になる。

5.6.5 基本例

V モーラのデフォルト表示は、次のような語で確認できる。

- yasai
 - 想定モーラ列: CV:ya, CV:sa, V:i
 - 表示: YaSaI
 - 語末 V モーラ i が大文字 I となる代表例である。
- tou
 - 想定モーラ列: CV:to, V:u
 - 表示: ToU

- 特定の語中例外には該当せず、語末 V モーラが デフォルト規則によって大文字化される例である。

ただし、すべての V モーラが常にこのデフォルト表示に従うわけではない。 **kitteiru**、**seisyohou**、**suu**、**toukyou**、**siteori**、**osaete**、**sieawo** などでは、後続の例外規則が優先される。

5.6.6 この節の位置づけ

本節で定めるのは、V モーラの標準的な表示である。実際の表示処理では、まず後続の V モーラ例外ルールを評価し、いずれにも該当しなかった場合にのみ、本節のデフォルト表示へフォールバックする。

したがって、本節は例外規則の対立項ではなく、すべての V モーラ表示を支える基準点として機能する。仕様変更時には、まずこの基準点が保たれているかを確認したうえで、各例外規則の挙動を検証する必要がある。### 5.7 V モーラ例外ルールの評価順

5.7 V モーラ例外ルールの評価順

前節で述べた「語中小文字／語末大文字」は、V モーラ表示の 基本形である。ただし実際の標準モードでは、すべての V モーラを このデフォルトだけで処理すると、拍の見え方が弱すぎたり、代表語で期待している独特の引っかかりが再現できなかったりする。そのため本仕様では、V モーラに対してはまず本節の例外規則を上から順に評価し、いずれにも該当しなかった場合にのみ 5.6 のデフォルト表示へ戻るものとする。

この評価順は、説明の都合で並べたものではなく、表示結果を安定させるための優先順位である。より限定的な条件を先に置き、より一般的な条件を後ろに置くことで、**kitteiru**、**seisyohou**、**suu**、**toukyou**、**siteori**、**osaete**、**sieawo** などの代表語と、V 系回帰テスト表の期待値が 同じ順序で再現されるように設計する。

5.7.1 kitteiru 系 (QCV + V + CV)

まず、促音を含む結合モーラの直後に単独母音 **i** が現れ、さらにその後ろに通常の子音＋母音モーラが続く場合は、その **i** を大文字 **I** として表示する。本仕様では、この型を V モーラ例外の最優先規則として扱う。

この規則は、代表例 **kitteiru** に対して適用される。想定モーラ列が **ki / tte / i / ru** のとき、**tte** の直後にある **i** がこの条件に該当し、最終表示は **KitTeIRu** となる。必須回帰テスト表では、1.1「Q 基本セット」に含まれると同時に、2.1「kitteiru 系 (ei 系／促音＋V)」の代表例としても 位置づけられている。

この規則を最優先に置く理由は、促音を含む塊の直後に来る母音を 強く立てることで、中央付近の拍のつながりと切れ目を 同時に見せやすくするためである。他の V 規則より先にこの条件を評価することで、**KitTeIRu** という本仕様の代表的な見え方を安定して再現できる。

- 代表例
 - **kitteiru** → **KitTeIRu**

5.7.2 ei 系一般化 (se / ke + V:i)

次に、直前のモーラが子音+母音モーラであり、その母音が **e**、かつ現在の単独母音モーラが **i** である場合は、その **i** を大文字 **I** として表示する。本仕様では、この型を「**e + i** 系一般化」と呼び、ei 系の語に共通して適用される V モーラ例外として扱う。

この規則は、**seisyohou**、**seikou**、**keiei**、**seion** などの代表語を対象とする。たとえば **seikou** では、想定モーラ列 **se / i / ko / u** に対して 2 拍目の **i** が **I** となり、表示は **SeIKoU** となる。同様に、**keiei** では **ke / i / e / i** の最初の **i** が立ち、**KeIEI** となり、**seion** では **se / i / o / N** の **i** が立ち、末尾の **N** は 'n' として表示されるため **SeIOn** となる。

この規則は、標準的なローマ字綴りを大きく崩しすぎずに、**e** に続く独立母音 **i** を「別拍として立てる」ための中核ルールである。後続の V 規則よりも先に評価することで、ei 系の語において **i** の拍が埋もれず、代表語テスト表が期待する表示を安定して得られるようにしている。

- 代表例
 - **seisyohou** → **SeISyoHoU**
 - **seikou** → **SeIKoU**
 - **keiei** → **KeIEI**
 - **seion** → **SeIOn**

5.7.3 suu（末尾 U を立てない）

次に、子音+母音モーラ **su** の直後に単独母音 **u** が置かれ、かつその **u** が語末にある場合は、語末であってもその **u** を大文字にせず、小文字のまま保持する。本仕様では、この型を V モーラの特例として扱い、**su + u** に限っては 5.6 の「語末大文字」ルールを上書きする。

この規則は、代表例 **suu** に対して適用される。想定モーラ列 **su / u** に対して、語末の V モーラ **u** をあえて小文字のまま据え置くことで、表示は **Suu** となる。V 系回帰テスト表では、「**V:u** (su の後、語末だが小文字のまま)」と明示されており、この挙動が仕様として固定されている。

もしこの規則がなければ、5.6 のデフォルト規則により語末の **u** は大文字 **U** となり、**SuU** のような強い見え方になる。しかし標準モードでは、**suu** に関しては連続する **u** の滑らかさを優先し、拍の終止感よりも一続きの伸びとして読めることを重視するため、この特例を導入している。

- 代表例
 - **suu** → **Suu**

5.7.4 toukyou（中間 U を立てる）

次に、単独母音 **u** が語中にあり、その前が **to**、その後ろに拗音を含むモーラが続く場合は、その **u** を大文字 **U** として表示する。本仕様では、この型を「中間 U を立てる」規則として扱い、**to + u + 拗音** の並びに対して優先的に適用する。

この規則は、代表例 **toukyou** に対して適用される。想定モーラ列が **to / u / kyo / u** のとき、2 拍目の V モーラ **u** がこの条件に該当し、表示は **ToUKyoU** となる。語末側の **u** については、5.6 のデフォルト規則により語末 V として大文字 **U** が用いられるため、結果として 2 つの **U** が現れる構成になる。

一方で、**tou** のように後続が拗音ではない場合には、この規則は適用されない。その場合、**to / u** の **u** は単に語末 V として扱われ、デフォルト規則により **ToU** となる。このように、**toukyou** では中間 U を立てる

例外規則が、**tou** では語末大文字というデフォルト規則が働くことで、似た綴りを持つ 2 語の拍の構造を視覚的に区別している。

- 代表例
 - **toukyou** → **ToUKyoU**
 - **tou** → **ToU**

5.7.5 siteori (eo)

次に、単独母音 **o** の直前に **te** がある場合は、その **o** を大文字 **O** として表示する。本仕様では、この型を eo 系の代表的な例外として扱い、**te + o** の並びにおいて後続の **o** を独立した拍として強く見せる。

この規則は、代表例 **siteori** に対して適用される。想定モーラ列が **si / te / o / ri** のとき、**te** の直後にある V モーラ **o** がこの条件に該当し、表示は **SiTeORi** となる。V 系回帰テスト表でも、この **o** は「V:o (te の後)」として評価対象に指定されている。

この規則の役割は、**eo** の並びを単なる連続母音として流さず、**te** のあとに別拍の **o** が立ち上がる感覚を明確に視覚化することにある。そのため、標準モードにおける eo 系の見え方を支える中核的な例外規則の一つとして位置づける。

- 代表例
 - **siteori** → **SiTeORi**

5.7.6 osaete (ae)

次に、語頭が単独母音 **o** で始まり、その後に **sa**、さらに単独母音 **e**、その後に通常モーラが続く場合は、語頭の **o** を大文字 **O**、**sa** の直後にある **e** を大文字 **E** として表示する。本仕様では、この型を ae 系の代表的な例外として扱い、語頭母音と中間母音の両方を立てる規則とする。

この規則は、代表例 **osaete** に対して適用される。想定モーラ列が **o / sa / e / te** のとき、語頭の **o** と中間の **e** がともに強調対象となり、表示は **OSaETe** となる。V 系回帰テスト表でも、この語については語頭 **V:o** と **V:e** の両方が評価対象として整理されている。

この規則は、**siteori** や **seikou** のように単一の V モーラだけを立てる規則よりも、やや強い演出である。標準モードでは、この語に対しては滑らかさよりも拍ごとの引っぱりを前面に出すことを優先し、ae 系の中でも例外性の強い代表例として位置づける。

- 代表例
 - **osaete** → **OSaETe**

5.7.7 sieawo (eo/ae 拡張候補)

次に、**e + a** が連続する拡張的な並びでは、先行する **e** を立てるだけでなく、後続の **a** も表示上の強調対象とする。本仕様では、この型を eo / ae 系の拡張候補として扱い、標準仕様の中心例というより、既存規則の外縁を確認するための補助的なルールとして位置づける。

この規則は、代表例 **sieawo** に対して適用される。想定モーラ列が **si / e / a / wo** のとき、**si** の後にある **e** と、その直後の **a** がともに評価対象となるため、表示は **SieAWo** となる。V 系回帰テスト表でも、この

語については **V:e** (**si** の後, **a** の前) に加え、後続の **V:a** も表示上の強調対象として扱う。

この規則は、既存の **siteori** や **osaete** を壊さずに eo / ae 系の拡張を導入できるかを観測するためのものである。したがって現時点では、完全に一般化済みの規則というよりも、代表語と回帰テストを通じて妥当性を継続確認する対象として扱う。

- 代表例
 - sieawo** → **SieAWo**

5.7.8 デフォルト V (語中小文字／語末大文字)

以上のいずれの例外規則にも該当しない単独母音モーラについては、5.6 で定めた基本形に戻り、語中では小文字、語末では大文字として表示する。本仕様では、このフォールバックを V モーラ表示の最終受け皿とし、未定義の語に対しても安定した表示を与えるための基準とする。

このデフォルト規則により、例外規則は必要な箇所だけに限定され、それ以外の V モーラは一貫した基準で処理される。たとえば **tou** は **to / u** というモーラ列を持つが、5.7.4 の「中間 U を立てる」規則には該当しないため、語末 V のデフォルト規則により **ToU** となる。同様に **yasai** は **ya / sa / i** とみなされ、末尾の **i** が語末 V として大文字化されるため **YaSaI** となる。

本仕様では、5.7.1 から 5.7.7 までの例外規則と、この 5.7.8 のデフォルト規則をあわせて V モーラ表示の全体像とみなす。したがって実装・改修時には、個別例外だけでなく、**tou** や **yasai** のようなデフォルト事例が崩れていないことも必ず併せて確認する必要がある。

- 代表例
 - tou** → **ToU**
 - yasai** → **YaSaI**

6. 章: 単語レベルの個別例外 (Sin'Ya / Kin'YoUBi)

6.1 位置づけと設計方針 (夜／曜日モチーフの強調)

ここで定める前処理の範囲は、以降のローマ字判定およびすばる式強調を適用するかどうかを決める前段階として機能する。

6.1.1 対象とする文字集合 (半角英数)

本ガイドラインにおいて、「ローマ字判定」および「すばる式強調」の対象とする文字は、半角英数のみとする。具体的には、次の文字集合を対象とする。

アルファベット (大文字・小文字)

A～Z

a～z

数字

0～9

アクセント付きアルファベット（例：é, ä）や、
全角アルファベット（例：A, B）、その他のマルチバイトの
ラテン文字は、本節で定める処理の対象外とする。

これらの文字は、入力文字列の一部としては残るが、
ローマ字判定やすばる式強調の判断には用いない。

また、日本語（ひらがな・カタカナ・漢字）や、
その他の記号類（句読点、括弧、記号など）も、
本ガイドラインが定義するローマ字判定・強調処理の対象外とする。

これにより、入力文字列に含まれる多言語・多文字種のうち、
すばる式が関与する範囲を「半角英数」に限定する。

6.1.2 単語の定義（スペース区切り）

本ガイドラインでは、「単語」を入力文字列のスペース区切り単位として定義する。具体的には、1 つ以上のスペース（空白文字）で分かれた各区間を 1 単位の「単語」とみなす。

例：

入力：foo bar baz

単語列：foo / bar / baz

入力：foo bar（スペースが連続する場合）

実装上の扱いは環境依存とするが、本ガイドラインでは
連続する空白を 1 つの区切りとして扱うことを推奨する。

本仕様では、入力の分かち書き（スペースによる区切り）そのものは
上位のシステムや前段の処理に委ねる。

すばる式強調ガイドラインは、あくまで「すでにスペースで分割された
単語列」を前提として、その各単語に対してローマ字判定・強調適用の
可否を決定する。したがって、本ガイドラインでは追加の「単語抽出処理」、
すなわち、入力文字列からラテン文字や記号のみを抜き出して
再構成するような処理は行わない。

単語境界は常に「もともとの入力におけるスペースの位置」に
依存するものとする。

6.1.3 半角英数以外を含む単語の扱い

1つの単語の中に、半角英数以外の文字（日本語、全角記号など）が混在する場合であっても、その単語はあくまで1単位として扱う。

ただし、ローマ字判定およびすばる式強調の判断は、その単語内の半角英数連続列を主な対象として行う。

たとえば、次のような入力を考える。

入力: foo 日本語 bar123

単語列: foo / 日本語 / bar123

このとき、foo および bar123 は半角英数を含む単語としてローマ字判定・強調対象候補となるが、日本語 は本ガイドラインの対象外として扱われる。

実際にどの単語が強調対象となるかは、6.2 以降で定めるローマ字綴り判定と適用条件に従う。

6.1.4 本節の位置づけ

本節で定めた「入力対象の制約」と「単語の定義」は、以降の章で扱うローマ字判定・強調処理の前提条件となる。特に、6.2 以降では次の前提を共有する。

ローマ字判定・すばる式強調の対象となるのは、半角英数を含む単語に限られる。

単語境界はスペース区切りであり、追加の単語抽出・再構成は行わない。

これにより、本ガイドラインは文字コードや多言語混在の問題を前処理段階で抱え込まず、半角英数に限定した範囲でローマ字綴り・モーラ強調の仕様を定義することに集中できる。

6.2 ローマ字綴り判定と混在許容ポリシー

本節では、各単語が「ローマ字綴りとして成立しているかどうか」を判定するためのポリシーを定義する。

ここでの判定結果は、すばる式強調を適用するかどうかを決めるためのトリガー判定として用いられ、厳密な正書法チェックを目的とするものではない。

6.2.1 判定の目的と非目的

ローマ字綴り判定の主な目的は、次のとおりである。

- すばる式強調を適用してよい単語かどうかを判断する。

- 明らかにローマ字とは無関係な英数字列（例: 長大な ID やランダムな英数字列）を、強調対象から除外する。
- 一方、この判定は次のような用途を目的としない。
 - ヘボン式・訓令式など、特定のローマ字体系への厳密な準拠判定。
 - 誤字・綴りミスの検出。
 - 日本語以外の言語（英語など）に対する正書法チェック。

したがって、本節で定義する判定は「ローマ字らしさ」を おおまかに確認するためのフィルタであり、完全な文法的妥当性を保証するものではない。

6.2.2 混在許容の基本方針

本ガイドラインでは、ヘボン式と訓令式の混在を許容する。すなわち、次のような単語はローマ字綴りとして成立し得るものとして扱う。

ヘボン式に近い綴り

例: shi, tsu, cha など

訓令式に近い綴り

例: si, tu, tya など

両者が混在した綴り

例: syu, jyo, ti など

これにより、実際の入力において
「ヘボン式で統一されていない」「訓令式が部分的に混ざっている」と
いったケースであっても、すばる式強調の適用対象から
過度に除外しない方針を採る。

ただし、どの程度まで混在を許容するかは、実装や運用方針に応じて
調整可能な余地を残す。

本ガイドラインでは、混在を前提とした上で、
最低限の「ローマ字らしさ」を確認するレベルに留めめる。

6.2.3 ローマ字らしさの判定単位

ローマ字綴り判定は、単語全体を対象として行う。単語は 6.1.2 で定義したとおり、スペース区切り単位である。

判定においては、単語中の半角英数連続列を主な対象とする。例えば、次のようなケースを考える。

foo

全体が半角英字で構成されているため、
ローマ字候補として判定対象となる。

bar123

先頭が英字、末尾が数字であるが、
「ローマ字らしい英字の並び」として扱うかどうかは
実装方針に委ねられる。

本ガイドラインでは、英字部分がローマ字として
解釈可能であれば候補として扱ってよい。

日本語foo

単語全体としては日本語と英字が混在するが、
ローマ字判定の対象となるのは foo の部分である。

実装上は、「半角英数部分のみを抽出して判定する」か
「単語全体としてローマ字ではない」とみなすかを
運用ポリシーに応じて選択できる。

本ガイドラインの標準的な解釈では、
「半角英字部分がローマ字として解釈できるかどうか」を
優先的に判定し、その結果をもって単語全体の
ローマ字綴り判定結果とすることを推奨する。

6.2.4 判定結果の扱い（トリガーとしての利用）

ローマ字綴り判定の結果は、次の 2 値として扱う。

ローマ字綴りとして成立する

ローマ字綴りとして成立しない この結果は、6.3 以降で定義する
「すばる式強調の適用条件」において、
次のように利用される。

単語がローマ字綴りとして成立しない場合

その単語はすばる式強調の対象外となり、
無変換（強調用の装飾なし）とする。

単語がローマ字綴りとして成立する場合

その単語は、すばる式強調の候補として
近傍単語との関係（6.3）や長大トークン判定（6.4）に
基づいて最終的に処理が決まる。

なお、この判定はあくまで「トリガーの ON/OFF」を決めるものであり、ローマ字として成立すると判定された単語が必ずすばる式強調の対象になるわけではない。
実際の適用可否は、6.3 および 6.4 の条件と組み合わせて判断す

6.2.5 今後の詳細化と拡張余地

本ガイドラインの本文では、ローマ字綴り判定の 詳細な受理パターン（例えば正規表現や音節表）についてはあえて踏み込まない。

これは、ヘボン式・訓令式・独自ローマ字など、運用環境ごとに異なる慣習を柔軟に取り込むためである。

具体的な判定ロジックは、次のいずれかの形で別途定義することを想定している。

付録として、受理パターン・禁止パターンを一覧形式で整理する。

実装仕様書側で、「ローマ字判定モジュール」の仕様として詳細を記述する。

本文側では、「混在を許容しつつ、ローマ字らしさをおおまかに確認する」という方針レベルを示すにとどめ、詳細実装に対して過剰な制約を課さない。

6.3 すばる式強調の適用条件（単語・文）

本節では、入力が 1 つの単語である場合と、複数の単語からなる文である場合とで、すばる式強調を適用するかどうかを判定する条件を定義する。

ここで定める条件は、6.1 の前処理と 6.2 のローマ字綴り判定を前提とした 最終的な適用ルールであり、実装上はこの節に従って強調対象単語の集合を 決定する。

6.3.1 共通前提条件

すばる式強調の適用可否は、次の前提条件を満たす単語に対してのみ 評価される。

単語は 6.1.2 で定義したスペース区切り単位である。

単語内に含まれる半角英数の連続列が、6.4 で定義する長大トークン閾値未満である。

単語が 6.2 のローマ字綴り判定において「成立する」と判定されている。これらの条件を満たさない単語については、すべて無変換とする。

特に、長大トークン（閾値以上の半角英数連続列を持つ単語）や、ローマ字綴りとして成立しない単語には、すばる式強調を一切適用しない。

6.3.2 分かち書きされていない 1 単語入力の場合

入力全体が 1 つの単語であり、内部にスペースを含まない場合は、次の手順で適用可否を判定する。

単語長の確認

単語内の半角英数連続列が、6.4 で定義する長大トークン閾値以上である場合、その単語は即座に無変換とする。

ローマ字綴り判定

上記を満たした場合、その単語が 6.2 において「ローマ字綴りとして成立する」と判定されているかどうかを確認する。

最終判定

ローマ字綴りとして成立する場合：
その単語をすばる式強調対象の単語とする。

ローマ字綴りとして成立しない場合：
その単語は無変換とする。

1 単語入力の場合、近傍単語との関係は考慮しない。
すなわち、入力全体が 1 語のみであれば、その語がローマ字綴りとして成立しているかどうかだけで、強調適用の可否を決定する。

分かち書きされた文の場合の近傍 5 語

入力が複数の単語からなる文であり、スペース区切りで分かち書きされている場合、本節では各単語に対して次のように「近傍 5 語」を定義する。

単語列を $w_0, w_1, \dots, w_{n-1}$ とする。

ある単語 w_i を評価対象（被変換単語）としたときの近傍は、次の 5 語とする。

w_{i-2} （2 つ前の単語）

w_{i-1} （1 つ前の単語）

w_i （被変換単語）

w_{i+1} （1 つ後の単語）

$w_{\{i+2\}}$ (2 つ後の単語)

先頭および末尾付近で、 $w_{\{i-2\}}$ や $w_{\{i+2\}}$ が存在しない場合は、その位置は単に存在しないものと扱い、判定から除外する。

例えば、 w_0 の 2 つ前、 w_1 の 2 つ前、 $w_{\{n-2\}}$ の 2 つ後、 $w_{\{n-1\}}$ の 1 つ後・2 つ後などは存在しない。

6.3.4 文中の単語に対する適用条件

分かち書きされた文においては、各単語 w_i について 次の手順で、すばる式強調を適用するかどうかを判定する。

長大トークン判定

w_i 内の半角英数連続列が、6.4 で定義する長大トークン閾値以上である場合、 w_i は即座に無変換とする。

自身のローマ字綴り判定

w_i が 6.2 において「ローマ字綴りとして成立しない」と判定されている場合、 w_i は無変換とする。

ローマ字綴りとして成立する場合のみ、次のステップに進む。

近傍単語のローマ字綴り判定

近傍 5 語のうち、 $w_{\{i-2\}}$ 、 $w_{\{i-1\}}$ 、 $w_{\{i+1\}}$ 、 $w_{\{i+2\}}$ の 4 語を対象に、それぞれがローマ字綴りとして成立するかどうかを確認する。

これら 4 語の中に、少なくとも 1 語でもローマ字綴りとして成立する単語が存在するかどうかを判定する。

最終判定

w_i 自身がローマ字綴りとして成立し、かつ近傍 4 語のいずれかがローマ字綴りとして成立する場合、 w_i をすばる式強調対象の単語とする。

w_i 自身はローマ字綴りとして成立しているものの、近傍 4 語すべてがローマ字綴りとして成立しない場合、 w_i は無変換とする。

このルールにより、文中の単語は「ローマ字らしい環境」の中に存在するときだけ、すばる式強調の対象となる。
単独で浮いている疑似ローマ字や、周囲が非ローマ字のみである単語については、強調を避けることで誤強調のリスクを抑制する。

6.3.5 この節の位置づけ

本節で定めた適用条件は、すばる式強調の適用範囲そのものである。具体的には、次のような役割を持つ。

どの単語が 5 章の表示分類仕様の対象となるかを決める。

7 章以降で構築する代表語リスト・回帰テスト表において、どの単語が強調確認の対象となるかの前提を与える。

長大トークンやローマ字から離れた英数字列に対して、強調処理を適用しないための安全装置として働く。

以降の章では、ここで「すばる式強調対象」と判定された単語だけが、モーラ構築・表示分類・強調装飾の詳細仕様の対象となる。

6.4 長大トークンの無変換閾値（32 文字＋オプション）

本節では、半角英数が異常に長く連続する単語（長大トークン）に対して、すばる式強調を適用しないための閾値を定義する。

この閾値は、ファイルチェックサムやハッシュ値などの用途で多用される 長い英数字列に対して、誤って強調装飾を適用しないための安全装置として機能する

6.4.1 デフォルト閾値（32 文字）

本ガイドラインでは、半角英数の連続列が 32 文字以上の単語を長大トークンとみなす。長大トークンと判定された単語は、すべて無変換（強調用の装飾を一切行わない）とする。

このデフォルト値 32 は、代表的なチェックサム・ハッシュのうち、特に MD5 の 32 桁 16 進表現を安全側で除外することを主な目的として設定している。

これにより、ファイルのチェックサムやハッシュ値を含む文章に対しても、デフォルト設定のままで、すばる式強調による視覚的な変形を避けることができる。

6.4.2 閾値設定オプション

実装および運用環境では、長大トークンを判定するための閾値を オプションとして変更できるものとする。

ここでいう閾値を L とし、「半角英数連続列の長さが L 文字以上なら長大トークンとみなす」という意味で用いる。

オプションとして設定された閾値 L は、すべての単語に対する 長大トークン判定に共通して適用される。

すなわち、1 単語入力・分かち書きされた文のいずれの場合も、同じ L を用いて長大トークンかどうかを判断する。

6.4.3 許容される値とデフォルトへのフォールバック

閾値オプション L に指定できる値は、次の条件を満たす整数とする。

$1 \leq L \leq 200$ これ以外の値が指定された場合は、デフォルト値 32 が指定されたものとみなす。
具体的には、次のような値はすべて 32 にフォールバックする。

負の値 ($L < 0$)

0

200 より大きい値 ($L > 200$)

数値として解釈できない値（非数値、パースエラー） 実装上は、次の動作を保証する。

有効な整数 L (1~200) が指定された場合

長大トークン閾値として、その値を用いる。

オプション未指定、あるいは無効な値が指定された場合

長大トークン閾値は 32 とする

6.4.4 判定ロジックへの組み込み

すばる式強調の適用可否を判定する際には、6.3 の各手順に先立って、次のように長大トークン判定を行う。

対象単語内の「半角英数字の連続長」を測定する。

ここで対象とするのは、ASCII の A~Z、a~z、0~9 の連続部分である。

その連続長が閾値 L (デフォルト 32) 以上である場合、

その単語は長大トークンとみなされ、即座に無変換とする。

この場合、ローマ字綴り判定や近傍判定は行わない。

連続長が L 未満である場合、

その単語については通常どおり 6.2 のローマ字綴り判定、6.3 の適用条件判定へ進む。

この順序により、長大な英数字列は常に先に除外され、ローマ字綴り判定や近傍 5 語ルールのような、ローマ字前提のロジックに巻き込まれないようになっている

6.4.5 本節の位置づけ

本節で定めた長大トークン閾値は、すばる式強調の安全装置として機能する。

特に、ファイルチェックサムやハッシュ値、長大な識別子などに対しては、デフォルト値 32 のまま運用することで、意図しない強調装飾を防ぐことができる。

なお、この閾値は「どの程度の長さから、もはや通常の単語とは見なさないか」という運用ポリシーにも依存するため、実装・運用環境に応じて L の値を調整してよい。

ただし、許容範囲外の値に対しては常に 32 へフォールバックするため、仕様としては常に安全側の挙動が保証される。

7. 章: 基本表示と表示例外

本ガイドラインでは、基本表示と表示例外を明確に分離する。

基本表示は、モーラ風トークン列と再束ね規則に基づいて機械的に生成される「すばる式ローマ字の標準出力」であり、表示例外は、公開上の字面調整や作品固有の演出のために後段で適用される追加ルールである。

基本表示は、5 章で定義したモーラ構築・再束ね・独立母音化の規則と、7.1～7.2 節で定める代表語一覧により、その挙動が本ガイドライン上固定される。

表示例外は、`display-exception-rules.json` 相当の辞書により管理され、語単位または複合語単位で標準出力を書き換える。

※ `nippon` については、本ガイドラインの代表語一覧では current 挙動 `NipPon` のみを掲載し、旧挙動 `Nippon` は **別途用意する回帰テスト計画ファイル側で versioned ケース (legacy) として管理する**。

実装・テストを行う AI / ツールは、「7 章の代表語表には current のみが載り、legacy はテスト計画ファイルに分離される」という前提を保持しなければならない。

※ `sieawo` についても、`nippon` と同様の方針を採用する。

本ガイドラインの代表語一覧（7.2 節）では、current 挙動である `SieAWo` のみを掲載し、旧挙動 `SieaWo` は **別途用意する回帰テスト計画ファイル側で versioned ケース (history / legacy) として管理する**。

実装・テストを行う AI / ツールは、「7 章の代表語表には常に current のみが載り、過去挙動や拡張案はテスト計画ファイルに分離される」という前提を、`sieawo` を含む V 系代表語にも適用しなければならない。

7.1 基本表示に関する代表語（概要）

本節では、基本表示の挙動を確認するための代表語セットのうち、N（撥音）、Q（促音）などを含む基本パターンについて概要を示す。

詳細な tokens / moras / result の対応は 5 章および 8 章（三段ログ）に委ねる。

（※7.1 の個別テーブルは既存ガイドラインを踏襲する想定のため、ここでは文面概要のみ）

7.1.1 代表語一覧（Q 基本セット）

word	category	tokens
result	test-table-ref / rules-applied / scope-flag / notes	
gakkou	Q-basic	CV:ga, Q:k, CV:ko, V:u
GakKoU	1.1, Q 結合+5.6, required, QCV 結合と語末 V デフォルト表示の基本例。	
toppu	Q-basic	CV:to, Q:p, CV:pu
TopPu	1.1, Q 結合+5.6, required, 2 拍目 ppu による子音強調 (pPu) の代表例。	
kitte	Q-basic	CV:ki, Q:t, CV:te
KitTe	1.1, Q 結合+5.6, required, 2 拍目 tte による子音強調 (tTe) の基本形。	
kitteiru	Q-basic+V-ei	CV:ki, Q:t, CV:te, V:i, CV:ru
KitTeIRu	1.1 / 2.1, 5.7.1, required, Q 基本セットかつ「QCV+V+CV」で V モーラ例外の代表。	
nippon	NQ-basic	CV:ni, Q:p, CV:po, N
NipPon	1.1 / NQ-N, 5.3.8, required, 語末 N を CVn に束ねて 2 拍目 `pP` を強調する current 表示。	

category は、Q 基本セットの中での役割を示すラベルである。

Q-basic+V-ei のように、Q 結合と V 例外が重なる代表語については、併記することを許容する。

rules-applied は、人間が読んだときに「どの規則が効いているか」を即座に思い出せるようにするための簡易ラベルであり、正式な規則 ID (例: 5.7.1) を優先して記載する。

7.1.2 この表の運用上の注意

本表に記載される代表語は、必須回帰テスト表 1.1 における「Q 基本セット」の行と一対一で対応する。

kitteiru については、V 系回帰テスト表 2.1 「kitteiru 系」とも対応関係を持つため、test-table-ref に 1.1 / 2.1 を併記している。

Q 結合の仕様や V モーラ例外 (5.7.1) を変更する場合は、必ず本表と対応する回帰テスト表の両方を更新し、7.5 の運用方針に従って整合性を確認すること。

7.2 V 代表語一覧（標準モード）

本節では、V モーラの挙動に関する代表語を一覧する。各行は、標準モードにおける出力と、その語が担う V 規則の カテゴリ・適用範囲を示す。

なお、本節に挙げる V 代表語は、回帰テスト計画ファイルにおける V 系回帰テストブロック（Block V）の必須テストケースと 一対一に対応している。

実装側では、ここに記載された input / result を V 系の基準期待値として扱う。

7.2.1 代表語一覧（長音・eo/ae セット）

word	category	tokens
result	test-table-ref / rules-applied / scope-flag / notes	
suu	V-long	CV:su, V:u
Suu	1.2 / 2.3, 5.7.3, required, 語末 v を立てない長音（su + u 特例）の代表例。	
oi	V-long-exception	V:o, V:i
Oi	1.2, 単語例外, required, 通常の v 規則とは別枠で扱う単語例外。	
tou	V-default	CV:to, V:u
ToU	1.2 / 2.4, 5.6（デフォルト）, required, 語末 v デフォルト表示の代表例。	
toukyou	V-long-U-middle	CV:to, V:u, CyV:kyo, V:u
ToUKyoU	1.2 / 2.4, 5.7.4 + 5.6, required, 中間 u を立てる規則と語末 v デフォルトが共存する代表例。	
yasai	V-default	CV:ya, CV:sa, V:i
YaSaI	1.2, 5.6（デフォルト）, required, 語末単独 v モーラ i を大文字化するデフォルトの代表例。	
seisyohou	V-ei-generalization	CV:se, V:i, CyV:syo, CV:ho, V:u
SeISyoHoU	1.2 / 2.2, 5.7.2, required, e + i 系一般化と語末 v の扱いを同時に確認する代表例。	
siteori	V-eo	CV:si, CV:te, V:o, CV:ri

SiTeORi	1.2 / 2.5, 5.7.5, required, te + o の eo 系例外で、3 拍目 o を立てる代表例。	
osaete	V-ae	V:o, CV:sa, V:e, CV:te
OSaETe	1.2 / 2.6, 5.7.6, required, 語頭 o と中間 e を立てる ae 系の強い演出代表例。	
seikou	V-ei-generalization	CV:se, V:i, CV:ko, V:u
SeIKoU	1.2 / 2.2, 5.7.2, required, e + i 系一般化と語末 u 表示が共存する代表例。	
keiei	V-ei-generalization	CV:ke, V:i, V:e, V:i
KeIEI	1.2 / 2.2, 5.7.2, required, e + i 系が複数回現れても拍の視認性を保つ代表例。	
seion	V-ei+N	CV:se, V:i, V:o, N
SeIOn	1.2 / 2.2, 5.7.2 + N→'n, required, e + i 系一般化と N→'n（撥音）両方を監視する複合代表例。	

7.2.2 この表の運用上の注意

本表に記載される代表語は、必須回帰テスト表 1.2「長音・eo/ae セット」の行と一対一で対応し、V 系回帰テスト表 2.2～2.6 の代表例とも対応付けられている。

seikou・keiei・seion は、5.7.2 の ei 系一般化を支える中核代表語であり、本文とテストの整合性を保つために scope-flag = required とする方針が確定している。

seion は、ei 系一般化と N→'n の両方に関わるため、8 章の三段ログ例（入力→moras→result）でも代表例として扱うことを想定している。

oi は「単語例外」として扱われるため、category を V-long-exception として区別している。必要に応じて scope-flag を required から extension に切り替えることもできる。

この一覧は、V 系挙動のうち標準モードで必須となる代表語のみを収録する。拡張仕様およびその履歴については、7.3 節で別途管理する。

7.3 V 例外拡張一覧

本節では、V モーラに関する拡張仕様と、その導入・昇格・廃止の履歴を一覧する。scope-flag は標準仕様との関係、phase は拡張の段階を示す。

- **extension**: 実装してもよいが、標準モードの必須要件ではない拡張仕様。
- **candidate**: 将来の標準仕様への昇格候補として検証中の仕様。
- **history**: すでに標準仕様へ昇格するなどして、拡張としては撤廃済みの履歴情報。

本節に記載する V 例外拡張および履歴情報は、回帰テスト計画における V 系 versioned テストおよび placeholder-guard テストの根拠となる。

とくに sieawo のように、拡張仕様から標準仕様へ昇格した語については、legacy 挙動（拡張時点の result）と current 挙動（標準代表としての result）の双方を追跡する必要があるため、本節の内容を変更した場合は必ず回帰テスト計画側も同期更新するものとする。

7.3.1 V 拡張・履歴一覧

input notes	result	v-pattern	scope-flag	phase
sieawo Ver 2.1 系で定義された V-ea-extension 代表。連母音塊 維持（`SieaWo`）として扱い、 「V-ea 系を基本表示側でどう 扱うか」を検証するための試験 拡張だった。Ver 2.2-draft1 で は `SieAWo` を V-ea 系の標準 代表として 7.2 節に昇格させた ため、この拡張形 `SieaWo` 自体 は拡張仕様として廃止され、本 ガイドライン上は履歴のみを保 持する。	SieaWo	V-ea	history	promoted-to- required
(placeholder-v-ea) (未定) 将来の V-ea 系拡張候補を一時 的に収容するためのプレースホル ダ行。具体的な語が提案されてい ない段階では、本ガイドライン上 も実装上も未使用とし、表示・変		V-ea	extension	extension

換口ジックには影響を与えない。					
実際の拡張提案例が現れた場合、					
この行の `input` / `result` /					
`notes` を具体語で置き換え、					
提案票・回帰テスト結果に基づき					
`scope-flag` / `phase` を更新する。					
(placeholder-v-	(未定)	V-other	candidate	candidate	V-
eo / V-ae / V-ea 以外の連続					母
other)					将
音 (例: `io`, `ua` など) が					一
来 v 特例として提案された際に					ッ
時的に登録するための候補スロ					装
ト。具体語が決まるまでは、実					
コードや表示例には反映せず、					
「候補種別としての枠組みのみ」					を
示す行として扱う。実際の候補					語
が現れた場合には、本行を具体					語
で置き換え、昇格プロセス					
(extension → candidate →					
required) に従って 7.2 節への					移
動や `history` 化を行う。					

7.4 N モーラ関連代表語

この節では、5.5 「N モーラ表示（概念 N と表示 'n）」に対応する代表語を、回帰テスト運用の観点から表形式で整理する。

本表は、8 章で説明する三段ログ（入力 → moras（概念） → result）と密接に連携し、N → 'n の固定変換が V モーラ例外と干渉せずに機能しているかを監視するための代表語一覧として位置づける。）

7.4.1 代表語一覧（N モーラ関連）

word	category	tokens
result	test-table-ref / rules-applied / scope-flag / notes	
seion	V-ei+N	CV:se, V:i, V:o, N
SeIOn	1.2 / 2.2, 5.7.2 + N→'n, required, ei 系一般化と N→'n（撥音）の両方を監視する複合代表例。	
nippon	NQ-basic	CV:ni, Q:p, CV:po, N
NipPon	1.1 / NQ-N, 5.3.8, required, 語末 N を CVn に束ねて 2 拍目 `pP` を強調する current 表示。	

category を V-ei+N とし、ei 系一般化（5.7.2）と N モーラ表示（5.5）が 同時に関与する語であることを明示している。

moras 列では、撥音モーラを概念記号 N で表し、result 列では末尾が必ず 'n に対応することを確認する。これは 5.5 で定義された「概念 N → 表示 'n」の二層構造と一致する。

7.4.2 三段ログとの関係

seion は、8 章で解説する三段ログの典型例としても用いられる予定である。三段ログにおいては、次のような対応関係を確認する。

入力: seion

tokens: CV:se, V:i, V:o, N

moras（概念）: se / i / o / N

result（表示）: SeIOn このとき、V モーラ例外（ei 系一般化）によって i が I として立ち上がり、同時に、末尾の N が 'n として表示される。

代表語リストでは、この挙動が仕様どおりに維持されているかどうかを
回帰テストで継続的に監視する役割を負う

7.4.3 今後の拡張余地

現時点では、N モーラ関連の代表語として seion を 中核的な監視対象として扱う。将来、他の N 含み語（例: ei 系以外の撥音語）を 代表語リストに追加する場合は、次の方針に従う。

追加する語が、どの表示規則（5.6 / 5.7.x）と $N \rightarrow 'n$ の組み合わせを監視するものを明記する。

8 章の三段ログ解説で利用する例が増えた場合は、本節の表と `test-table-ref` を合わせて更新する。

`scope-flag` を `required` とするか `extension` とするかは、その語が必須回帰テスト表に含まれるかどうかに合わせて決定する。

こうしておくことで、 N モーラ関連の仕様が拡張された場合でも、代表語リストと三段ログの両方を通じて挙動の一貫性を確認しやすくなる。

7.5 代表語リストの運用方針

本節では、本章に掲載する代表語リストの役割と運用ルールを定義する。

代表語リストは、5 章の表示分類仕様および `required-regression-test-plan.md` に記載される回帰テスト表と密接に結び付いたドキュメントであり、単なる例示集ではなく「仕様とテストの対応関係を管理するための一覧」として位置づける

7.5.1 代表語リストの役割

代表語リストは、次の役割を担う。

5 章の各節で定義された表示規則（デフォルトおよび例外）の代表的な適用例を、表形式で一覧化する。

回帰テスト表 1.x（代表語テスト表）および 2.x（ v 系回帰テスト表）に記載された行と、本文中の説明を相互に参照しやすくする。

実装および改修時に、どの語がどの規則の代表例として監視されるべきかを、明示的に管理する。したがって、本章に掲載される代表語は、5 章本文における「代表例」の再掲であると同時に、回帰テスト表における「必須確認項目」の一覧でもある。

7.5.2 6 章との関係（適用スコープ）

代表語リストに掲載される語は、6 章で定義した すばる式強調の適用条件を満たすことを前提とする。

単語がローマ字綴りとして成立していること（6.2）。

長大トークン閾値（6.4）の範囲内であること。

分かち書きされた文においても、近傍 5 語の条件を満たす環境で強調対象となり得ること (6.3)。この前提により、代表語リストは「理論上は強調対象となる語」のみを集約した一覧となる。

6 章のスコープ外にある語 (例: 長大ハッシュ値や明らかな非ローマ字列) は、代表語リストの対象外とする。

7.5.3 回帰テスト表との同期ルール

代表語リストと回帰テスト表は、常に同期された状態を保つことを基本ルールとする。

代表語リストに新しい語を追加する場合は、対応する回帰テスト表 (1.x / 2.x) の行も追加する。

回帰テスト表から語を削除する場合は、代表語リストからも同じ語を削除するか、少なくとも「非推奨」「廃止予定」などのステータスを明示する。

代表語リストと回帰テスト表のどちらか一方だけを変更した状態は、仕様として「不整合」とみなす。実運用では、回帰テスト表を更新する際に 7 章の該当行も同時に更新する運用フローを推奨する。

これにより、仕様書とテスト資産の乖離を防ぎ、第三者レビューやメンテナンス時に一貫した判断ができるようにする。

7.5.4 scope-flag の運用 (必須／拡張)

本章の代表語リストでは、各行に scope-flag (スコープフラグ) を付与することを推奨する。

scope-flag は、その語がどのレベルで必須かを示す指標である。

required

標準仕様において必須の代表語。

回帰テスト表においても必ず実行されるべきテストケースとして位置づける。

例: seikou, keiei, seion など (ei 系一般化の中核代表語)。

extension

標準仕様の外側にある拡張候補。

実装・運用環境によってはテスト対象外とする運用も許容する。

例: sieawo (eo/ae 拡張候補)。

experimental

実験的・観測中の代表語。

挙動の妥当性を検証中であり、将来 `required` または `extension` に格上げ／格下げされる可能性がある。

`scope-flag` を用いることで、9 章で定義する拡張仕様・保留仕様と代表語リストの関係を整理しやすくなる。

特に `sieawo` のような語については、5 章・7 章・9 章のどこに位置づけるかを一貫して管理するために、このフラグの活用を前提とする。

7.5.5 代表語リストの拡張と保守

代表語リストは、仕様と実装の両方の変化に合わせて段階的に拡張・保守されることを想定する。

新しい表示規則（例：新たな `v` 例外）が 5 章に追加された場合、その規則を代表する語を 7 章に 1 行以上追加する。

既存の代表語の扱いが変更された場合（例：拡張仕様から標準仕様への昇格）、`scope-flag` と `test-table-ref` を更新する。

不要になった代表語は、削除するか、「廃止」「歴史的経緯」として別枠に移すこともできる。これらの変更は、必ず 5 章の本文・回帰テスト表・9 章の拡張仕様とセットでレビューすることを推奨する。
代表語リストの単独更新は避け、常に仕様全体との整合性を確認したうえで変更する。

8. 章: 三段ログと回帰テスト運用

8.1 三段ログの定義とフォーマット

本章では、本ガイドラインで定義したモーラ構築・再束ね・基本表示の規則が、実際の入力列に対してどのような字面になるかを、代表的な例により示す。

各行の「入力」および「基本表示または代表表示」は、5章および7章で定めた仕様の具体例であり、別ファイルの回帰テスト計画におけるテストケース ID と一対一に対応する。

8.1.1 三段ログの役割

三段ログは、次の目的で用いられる。

4 章のモーラ構築仕様と 5 章の表示分類仕様のあいだにある中間状態を、具体的な例に基づいて可視化する。

7 章の代表語リストに挙げられた各語について、
入力 → モーラ列（概念） → 最終表示の対応関係を
一貫して記録する。

required-regression-test-plan.md に記載された回帰テスト表の
各行が、仕様上どのような変換経路を通っているかを
再確認するための資料として機能する。これにより、仕様変更や実装変更が行われた際に、
入力・モーラ列・表示のいずれかに不整合が生じていないかを
三段ログを通じて点検できる。

8.1.2 必須列と補助列

三段ログは、次の必須列を持つ。

input

すばる式強調の適用対象となる入力単語を、そのまま記録する。

6 章の適用条件を満たした単語（ローマ字判定済み・長大トークン除外済み）を
対象とする。

moras（概念）

4 章で定義したモーラ構築の結果を、概念ラベル列として記録する。

CV / CyV / V / QCV / N などのモーラ単位を、
スラッシュ区切りなど一貫した形式で表す。

撥音モーラは、常に概念記号 N で表記する（5.5）。

result

5 章の表示分類仕様と V モーラ例外（5.6・5.7）に従って得られた
最終表示を記録する。

撥音モーラは、常に 'n（アポストロフィ+小文字 n）で表記する（5.5）。

必須列に加え、次の補助列を持つことを推奨する。

tokens

input から moras を構築する際に用いるトークン列。

4 章で用いている表現（例: CV:ki, Q:t, CV:te, V:i, CV:ru）を
そのまま記録する。

rules-applied

moras から result への変換において適用した表示規則の一覧。

5.6（デフォルト V）や 5.7.1～5.7.7（V 例外）、

および $N \rightarrow 'n$ の変換 (5.5) などを識別できるよう、
節番号や簡潔なラベルで記録する。

test-table-ref

7 章の代表語リストおよび回帰テスト表 (1.x / 2.x) の
該当行を指す参照 ID。

例: 1.1、1.2 / 2.2、2.7 など。

8.1.3 列フォーマットの例

典型的な三段ログ行の列構成は、次のようになる。

input

例: seion

tokens

例: CV:se, V:i, V:o, N

moras

例: se / i / o / N

result

例: SeIOn

rules-applied

例: 5.7.2 + $N \rightarrow 'n$

test-table-ref

例: 1.2 / 2.2

同様に、kitteiru の場合は次のように記録される。

input: kitteiru

tokens: CV:ki, Q:t, CV:te, V:i, CV:ru

moras: ki / tte / i / ru (または CV:ki, QCV:tte, V:i, CV:ru)

result: KitTeIRu

rules-applied: 5.7.1

test-table-ref: 1.1 / 2.1 このように、三段ログは 5 章・7 章で既に定義された情報 (tokens / moras / result / test-table-ref) を再利用しつつ、どの表示規則が適用されたかを明示するための枠組みとして設計されている。

8.1.4 本フォーマットの前提

三段ログのフォーマットは、次の前提に基づいて設計されている。

6 章で定義した適用スコープ（ローマ字判定・長大トークン除外）を満たした単語のみを対象とする。

4 章のモーラ構築仕様に従って tokens・moras を決定し、
5 章の表示分類仕様に従って result を決定する。

7 章の代表語リストと test-table-ref を通じて
回帰テスト表と対応付けられる。以降の 8.2 では、本フォーマットを前提として
5 章・7 章との対応を具体的に整理し、
8.3 では回帰テスト運用の中でこの三段ログを
どのように利用するかを手順として定義する。

8.2 三段ログと 5 章・7 章の対応

本節では、三段ログが 5 章の表示分類仕様および 7 章の代表語リストとどのように対応するかを整理する。

三段ログは、4 章のモーラ構築から 5 章の表示分類、さらに 7 章・回帰テスト表に至るまでの経路を、1 行のレコードとして具体化したものである。）

8.2.1 5 章（表示分類仕様）との対応

5 章では、V モーラのデフォルト表示 (5.6) と 各種例外規則 (5.7.1～5.7.7)、および N モーラ表示 (5.5) が節ごとに定義されている。

三段ログでは、これらの規則が各代表語に対してどのように適用されているかを、moras・result・rules-applied の 3 列を通じて確認する。

V モーラ表示 (5.6・5.7.x)

moras 列では、V モーラを含む拍構造を
CV / CyV / V / QCV などの単位で記録する。

result 列では、5.6 のデフォルトルール
「語中小文字／語末大文字」および
5.7.x の例外ルールによる大文字化の結果を記録する。

rules-applied 列には、どの節の規則が
moras → result の変換に関与したかを
5.6、5.7.2 のような形で記録する。

N モーラ表示 (5.5)

moras 列では、撥音モーラを常に N で表記する。

result 列では、対応する位置を必ず 'n として表記する。

rules-applied には、N→'n あるいは
「5.5 (N モーラ表示)」のような形で
N → 'n の固定変換が適用されたことを示す。

これにより、5 章で文章として定義された規則が、
各代表語に対してどのように実現されているかを
三段ログ上で直接確認できる。

8.2.2 7 章 (代表語リスト) との対応

7 章では、代表語ごとに word・tokens・moras・result・test-table-ref・scope-flagなどを表形式で整理している。

三段ログは、この代表語リストの各行をそのまま 変換経路付きのレコードとして拡張したものみなせる。

対応関係の基本

7 章の word は、三段ログの input に対応する。

7 章の tokens は、三段ログの tokens に対応する。

7 章の moras (概念)・result は、
三段ログの moras・result にそのまま対応する。

7 章の test-table-ref は、三段ログの
test-table-ref 列として引き継がれる。

三段ログ側の追加情報

三段ログでは、rules-applied 列を追加することで、
5.6・5.7.x・5.5 のどの規則が適用されたかを
代表語リストよりも明確に示す。

必要に応じて、scope-flag や category も
追加列として持たせることができる。

この対応により、7 章の代表語リストと三段ログは
「同じ行を、用途別にビューを変えたもの」として扱うことができる。

8.2.3 具体例: seion (V-ei + N)

seion は、ei 系一般化 (5.7.2) と $N \rightarrow 'n$ (5.5) の両方が関わる複合代表例である。

この語に対する三段ログは、7.2・7.4 の記述と 次のように対応する。

7 章での情報 (抜粋)

```
word: seion
tokens: CV:se, V:i, V:o, N
moras: se / i / o / N
result: SeIOn
test-table-ref: 1.2 / 2.2
category: V-ei+N
scope-flag: required
```

三段ログでの対応

```
input: seion
tokens: CV:se, V:i, V:o, N
moras: se / i / o / N
result: SeIOn
rules-applied: 5.7.2 +  $N \rightarrow 'n$ 
test-table-ref: 1.2 / 2.2
```

ここで確認したいポイントは次の通りである。

V モーラ例外 (5.7.2)

se / i / o / N において、se の直後の V モーラ i が ei 系一般化により I として立ち上がっていること。

$N \rightarrow 'n$ (5.5)

moras の末尾 N に対応する result の末尾が 'n で表記されていること。

三段ログでは、これら 2 つの規則が衝突せずに同時に適用されていることを、1 行のレコードとして明示的に確認できる。

8.2.4 具体例: kitteiru (QCV + V + CV)

kitteiru は、Q 基本セット (5.4.1) かつ V モーラ例外 5.7.1 「kitteiru 系」の代表例である。

7 章での情報 (抜粋)

```
word: kitteiru

tokens: CV:ki, Q:t, CV:te, V:i, CV:ru

moras: CV:ki, QCV:tte, V:i, CV:ru

result: KitTeIRu

test-table-ref: 1.1 / 2.1

category: Q-basic+V-ei

scope-flag: required
```

三段ログでの対応

```
input: kitteiru

tokens: CV:ki, Q:t, CV:te, V:i, CV:ru

moras: ki / tte / i / ru (または CV:ki, QCV:tte, V:i, CV:ru)

result: KitTeIRu

rules-applied: 5.7.1

test-table-ref: 1.1 / 2.1
```

ここで確認したいポイントは次の通りである。

Q モーラ結合

Q:t と CV:te が結合して QCV:tte となり、
moras 列で 2 拍目の結合モーラとして表現されていること。

V モーラ例外 (5.7.1)

ki / tte / i / ru において、tte の直後の V モーラ i が
5.7.1 の条件 (QCV + V + CV) を満たし、I として立ち上がっていること。

デフォルト V (5.6) との関係

語末の V (ru の u) は、5.6 のデフォルト V 表示
「語末 V を大文字」に従って U になるかどうかは
他のルールとの兼ね合いで決まるが、

三段ログでは rules-applied を通じて
5.7.1 が優先されていることを確認できる。

このように、三段ログは q 結合・v 例外・デフォルト v の
評価順（5.7 全体）を、代表語レベルで検証するための
具体的な記録となる。

8.2.5 表示例（標準モード・代表）

8.2.5 表示例（標準モード・代表）

```text

| 入力       | 基本表示または代表表示 | 備考                                                                            |
|----------|-------------|-------------------------------------------------------------------------------|
| nippon   | NipPon      | 語末 n は独立化しない。語末 N を CVn として直前塊へ従属させる例。                                        |
| kensaku  | KenSaKu     | ken を CVn として束ねる例。撥音再束ねと cv の大文字化が同時に現れる（Block NQ, TC-NQ-N-01 関連）。            |
| enzin    | EnZin       | en を Vn として束ねる例。Vn と CVn が混在するケース（Block NQ, TC-NQ-N-02/03 関連）。                |
| toppuno  | ToppuNo     | 促音を後続塊へ吸収し、その後に CVn を形成する例（Block NQ, TC-NQ-Q-03）。                             |
| kitteiru | KitteIRu    | 促音塊確定後に後続母音を独立化する例。Q+CV 吸収と v の独立母音化が連続して適用される（Block NQ / Block V 両方にまたがる代表）。 |
| gakkou   | GakkoU      | 語中の促音と語末 ou 独立母音化を併せ持つ例。Q+CV 吸収後、語末 ou を oU として立てる（Block NQ / Block V）。       |
| suu      | Suu         | 同母音長音の代表。CV + 同母音 v を長音塊として保持する（Block V, TC-V-LONG-01）。                       |
| oi       | Oi          | 一般二重母音 oi の代表。特別扱いせず維持寄りに扱う（Block V, TC-V-LONG-02）。                           |
| tou      | Tou         | 語中 ou 長音寄り既定値の代表（Block V, TC-V-LONG-03）。                                      |
| yasai    | YaSaI       | 語末 ai 独立母音化代表、日本語「やさい」。語末 ai を aI として分割し、違和感を立てる（Block V, TC-V-POS-01）。       |
| gyoukai  | GyoUKaI     | 語中 ou+語末 ai の独立母音                                                             |

|              |              |                             |
|--------------|--------------|-----------------------------|
|              |              | 化代表、日本語語「業界」。               |
|              |              | GyoU / KaI の両方で母音を立て        |
|              |              | る (Block V, TC-V-POS-02)。   |
| kaihatu      | KaIHaTu      | 語中 ai 独立母音化代表、日本            |
|              |              | 語語「開発」。語頭 Kai を             |
|              |              | 二重母音として維持せず、KaI             |
|              |              | へ分割する (Block V,             |
|              |              | TC-V-POS-03)。               |
| seisyohou    | SeISyoHoU    | 語中 ei+語末 ou の独立母音           |
|              |              | 化代表。SeI / HoU のように、         |
|              |              | 長音より仮名塊の露出を優先す              |
|              |              | る (Block V, TC-V-POS-04)。   |
| siteori      | SiTeORi      | V-eo 系独立母音化代表、日本            |
|              |              | 語語「しており」。TeO 部分で            |
|              |              | V-eo を独立母音として立てる            |
|              |              | 特例 (Block V, TC-V-SPEC-01)。 |
| osaete       | OSaETe       | V-ae 系独立母音化代表、日本            |
|              |              | 語語「おさえて」。SaE 部分で            |
|              |              | V-ae を独立母音として立てる            |
|              |              | 特例 (Block V, TC-V-SPEC-02)。 |
| sieawo       | SieAWo       | V-ea 系独立母音化代表。Ver           |
|              |              | 2.2-draft1 では SieAWo を      |
|              |              | 標準代表とし、旧挙動 SieaWo           |
|              |              | は 7.3 節および回帰テスト計画           |
|              |              | の versioned テストで履歴と         |
|              |              | して保持する (Block V,            |
|              |              | TC-V-SPEC-03)。              |
| eigo・ratengo | EIGo・RaTenGo | 複合語単位で例外を与える例。              |
|              |              | 基本表示に対して                    |
|              |              | display-exception を適用する     |
|              |              | 代表 (Block EX)。              |

### 8.2.6 まとめ（対応関係の位置づけ）

三段ログは、次の対応関係を明示するためのフォーマットである。

5 章の各表示規則が、どの moras → result の変換に対応するか。

7 章の代表語リストの各行が、どの三段ログ行と一致するか。

回帰テスト表の各行が、どの input / moras / result を前提として設計されているか。以降の 8.3 では、この三段ログをテスト運用の中でどのように参照・更新するかを手順として定義する。

## 8.3 回帰テスト表との連携手順

本節では、三段ログと required-regression-test-plan.md（必須回帰テスト表）をどのように連携させて運用するかを、具体的な手順として定義する。



ここで定める手順は、仕様変更・実装変更の際に 回帰テスト表と仕様書本文が常に同期した状態を維持することを 目的とする。

### 8.3.1 前提と対象

本手順の対象は、次の要素で構成される。

7 章に記載された代表語リスト (7.1~7.4)

8.1・8.2 で定義された三段ログ (input / tokens / moras / result など)

required-regression-test-plan.md に記載された  
代表語テスト表 1.x および V 系回帰テスト表 2.x。これらは次の対応関係を持つ。

7 章の各行 ⇔ 三段ログの各行 ⇔ 回帰テスト表の各行 したがって、いずれか 1 つを変更する場合でも、  
残り 2 つと整合が取れているかを必ず確認する必要がある。

### 8.3.2 通常運用時の連携手順

通常の運用（仕様変更や改修がない状態）で、回帰テスト表を参照・実行する際の基本手順は次のとおりとする。

#### 代表語リストの確認

7 章から、対象とする代表語セット（例: q 基本セット、  
長音・eo/ae セット）を特定する。

必要に応じて scope-flag を参照し、  
required に分類されている語を優先的にテスト対象とする。

#### 三段ログの参照

対象代表語について、三段ログにおける  
input / tokens / moras / result / rules-applied を確認する。

特に moras と result の対応、および  
適用される表示規則 (5.6 / 5.7.x / N→'n) を再確認する。

#### 回帰テスト表との照合

test-table-ref を用いて、回帰テスト表 1.x / 2.x の  
該当行を特定する。

回帰テスト表に記載された moras (概念) 列  
および result (期待値) 列が、三段ログと一致しているか  
を確認する。

不一致がある場合は、仕様書・テスト表のどちらが最新版かを  
確認したうえで、7.5 の運用方針に従い同期を取る。

## テスト実行

回帰テスト実行時には、入力 (input) と  
期待される result をテストケースとして使用する。

必要に応じて、ログやデバッグ出力に  
moras・rules-applied を含めることで、  
挙動が仕様どおりであることを検証する。

### 8.3.3 仕様変更時の連携手順

5 章・6 章・7 章いずれかの仕様を変更する場合は、次の順序で三段ログと回帰テスト表を更新することを推奨する。

#### 仕様変更点の特定

変更対象がどの規則か（例：5.7.2 の条件、6.3 の適用条件、  
7 章の代表語入れ替えなど）を明確にする。

#### 三段ログの更新

変更対象となる代表語について、  
新しい仕様に従って tokens / moras / result / rules-applied を  
三段ログ上で更新する。

特に N → 'n と V 例外の組み合わせ（例：seion）の場合は、  
moras・result の両方を慎重に見直す。

#### 代表語リスト（7 章）の更新

三段ログの更新内容に合わせて、7 章の  
tokens・moras・result・category・scope-flagなどを更新する。

代表語の追加・削除がある場合は、7.5 の運用方針に従って  
scope-flag や test-table-ref を調整する。

#### 回帰テスト表の更新

test-table-ref に基づき、回帰テスト表 1.x / 2.x の  
該当行を更新する。

moras（概念）列と result（期待値）列が、  
更新後の三段ログと一致するように修正する。

必要に応じて、テストケースの説明欄に  
簡単なコメント（例：「5.7.2 条件変更に伴う更新」）を追加する。

## 整合性チェック

更新後、少なくとも次の三点が一致しているかを確認する。

5 章の本文に書かれた規則の説明

7 章の代表語リスト

三段ログおよび回帰テスト表の内容

不一致が残っている場合は、仕様変更の方針を再確認し、どの文書を「正」とするかを明確にしたうえで再修正する。

### 8.3.4 N → 'n と V 例外に関する特別な注意

N モーラおよび V モーラ例外が同時に関わる代表語（例: seion）については、次の点に特に注意する。

回帰テスト表の moras（概念） 列では、撥音を必ず N として記録する。

同じ行の result（期待値） 列では、対応する位置を必ず 'n として記録する。

三段ログの rules-applied に、V 例外規則（例: 5.7.2）と N → 'n の変換が両方含まれていることを確認する。これにより、実装側で N → 'n の変換を変更した場合でも、回帰テスト表と三段ログの両方を通じて挙動の変更が検出しやすくなる。

### 8.3.5 非推奨・拡張テストの扱い

scope-flag が extension または experimental に設定された 代表語（例: sieawo）については、次のように回帰テスト表と連携する。

デフォルト運用では、required の代表語に対応するテストケースのみを必須回帰として実行し、extension・experimental は任意実行とする。

拡張仕様を有効にした環境では、extension の行も required と同等に扱うことができる。

回帰テスト表側では、scope-flag に対応するカラムやコメント欄を用いて、テストケースが必須か任意かを明示することを推奨する。こうしておくことで、標準仕様の回帰テストと拡張仕様の回帰テストを、運用上切り分けやすくなる。

## 8.4 三段ログの更新・拡張ポリシー

本節では、三段ログの行および列をどのように更新・拡張するかのポリシーを定義する。

このポリシーは、5〜7 章の仕様変更や回帰テスト表の更新と連動して 三段ログを保守する際の判断基準として機能する。

### 8.4.1 行の追加・削除ポリシー

三段ログの各行は、7 章の代表語リストおよび 回帰テスト表の各行と一対一に対応する。

したがって、行の追加・削除に関しては次のルールに従う。

#### 行の追加

5 章に新しい表示規則（デフォルト・例外いずれも）が追加された場合、その規則を代表する語を 7 章に追加し、三段ログにも対応する行を追加する。

このとき、回帰テスト表 1.x / 2.x にも新しいテストケース行を追加し、`test-table-ref` を三段ログに設定する。

#### 行の削除

仕様上不要になった代表語（例：置き換えられた旧代表語）については、7 章・三段ログ・回帰テスト表のすべてから削除するか、7.5 のポリシーに従い「廃止」「歴史的経緯」として別枠に移す。

いずれの場合も、三段ログだけが古い代表語を保持し続ける状態は許容しない。

#### 行の無効化

一時的に使用を停止する代表語については、削除ではなく `scope-flag` を `experimental` などに変更し、テスト表側でも同様のステータスを付与する。

三段ログ上では、その行を残したまま、ステータス列（拡張・実験・廃止予定など）で状態を明示する。

### 8.4.2 列の拡張ポリシー

三段ログの列は、8.1 で定義した必須列（`input` / `moras` / `result`）と 補助列（`tokens` / `rules-applied` / `test-table-ref`）を基本構成とする。

列の追加・変更については、次の方針を採る。

### 必須列の扱い

`input・moras・result` の 3 列は、三段ログの最小構成として必ず維持する。

これらの列名・意味を変更することは原則として禁止とし、仕様変更があっても列の意味は変えない。

### 補助列の追加

新たなメタ情報が必要になった場合  
(例: ロケール、入力種別、テストカテゴリなど)、補助列として新しい列を追加してよい。

追加する列は、7 章や回帰テスト表と可能な範囲で列名・意味を揃えることを推奨する。

### 補助列の削除・統合

補助列の削除や統合を行う場合は、7 章および回帰テスト表側の列構成にも影響が及ぶため、仕様全体をレビューしたうえで実施する。

特に `rules-applied・test-table-ref` のような追跡性に関わる列は、安易に削除・統合しない。

## 8.4.3 標準仕様と拡張仕様の区別

三段ログには、標準仕様に属する代表語と、拡張・保留項目に属する代表語の両方が含まれ得る。

これらを区別するために、次の方針を採る。

### `scope-flag` の活用

各行に `scope-flag` (`required` / `extension` / `experimental` など) を付与し、標準仕様と拡張仕様の境界を明示する。

9 章で定義される拡張・保留項目に対応する行には原則として `extension` または `experimental` を設定する。

### 回帰テストとの関係

標準仕様に属する行 (`scope-flag` が `required`) は、必須回帰テスト表に対応させる。

拡張仕様に属する行 (`scope-flag` が `extension` など) は、回帰テスト表側でも拡張カテゴリとして扱い、実行するかどうかを運用ポリシーで決定する。

### 昇格・降格の扱い

拡張仕様が標準仕様に昇格する場合は、  
scope-flag を extension から required に変更し、  
回帰テスト表を必須枠に移す。

逆に、標準仕様から外したい場合は、  
scope-flag を extension 等に変更し、  
必須回帰テストから外す。

## 8.4.4 変更履歴とレビュー

三段ログは、仕様と実装の両方に直結するため、変更時には必ずレビューと履歴管理を伴うべきとする。

### 変更履歴

三段ログの大きな変更（行の追加・削除、  
列構成の変更など）については、  
変更理由と対象行・対象列を簡潔に記録する。

記録先は、本ガイドラインの付録、  
または別途管理される変更履歴文書とする。

### レビュー

三段ログの変更は、5 章・7 章・回帰テスト表の  
いずれかを触ったタイミングで、  
仕様担当者とテスト担当者の両方によるレビューを前提とする。

7.5 および 8.3 で定めた運用方針に従い、  
代表語リスト・回帰テスト表との整合性が保たれているかを  
確認する。

## 8.4.5 本節の位置づけ

本節で定めた更新・拡張ポリシーは、三段ログを「一度作ったら終わりの静的な表」ではなく、仕様・実装・テストの変化に合わせて 継続的にメンテナンスされるべき「生きたドキュメント」として 扱うための指針である。

## 9. 章: 拡張・保留項目（将来モード用メモ）

### 9.1 拡張仕様の位置づけ

本章では、これまでの章で定義した標準仕様の外側に存在する 拡張候補・保留項目を整理する。

ここでいう「拡張仕様」とは、現時点では標準仕様には含めないが、将来的に導入する可能性があるルールや代表語、あるいは運用環境によって有効・無効を切り替えることを想定した追加仕様を指す。

### 9.1.1 標準仕様と拡張仕様の境界

本ガイドラインでは、次のように標準仕様と拡張仕様を区別する。

#### 標準仕様

5 章までに定義された表示分類仕様（N モーラ表示、V モーラのデフォルト表示および V 例外）と、6 章の適用スコープ、7 章の代表語リスト、8 章の三段ログおよび回帰テスト運用のうち、scope-flag が required とされている部分を指す。

#### 拡張仕様

scope-flag が extension または experimental とされている代表語・規則を指す。

これらは、デフォルト運用では必須ではなく、環境に応じて有効・無効を切り替えられる余地を持つ。

### 9.1.2 scope-flag による層別

7 章および三段ログでは、各代表語・各行に対して scope-flag（スコープフラグ）を付与する方針を取っている。拡張仕様は、この scope-flag を通じて標準仕様から区別される。

#### required

標準仕様に含まれる代表語・テストケース。

5 章本文と回帰テスト表の両方で必須とされる。

#### extension

拡張仕様には属する代表語・テストケース。

デフォルト運用では任意実行とし、拡張機能を有効にした環境で必須扱いにすることもできる。

例：sieawo を代表とする eo/ae 系拡張候補。

#### experimental

実験的・検証中の仕様に属する代表語・テストケース。

将来的に required または extension に

昇格・降格される可能性がある。

9 章で扱う拡張・保留項目は、原則として  
extension または experimental に分類される。

### 9.1.3 拡張仕様の適用と非適用

拡張仕様の適用有無は、実装・運用環境ごとに決定できるものとする。

標準運用（拡張仕様を無効とする場合）

scope-flag が required の行に対応する  
表示規則・代表語・テストケースのみを有効とし、  
extension・experimental の行は実装・テストともに  
任意扱いとする。

拡張運用（拡張仕様を有効とする場合）

scope-flag が extension の行も required と同等に扱い、  
拡張仕様を含めた形で表示結果とテスト範囲を定義する。

この場合、5～8 章の該当する規則・代表語・三段ログが  
すべて有効化されている必要がある。

いずれの場合も、拡張仕様の適用状態は  
実装仕様書や運用ドキュメントで明示することを推奨する。

### 9.1.4 本節の位置づけ

本節で定めた「拡張仕様の位置づけ」は、9.2 以降で扱う具体的な拡張候補（特に V モーラの eo/ae 系拡張や、ローマ字判定・適用スコープの拡張候補）を整理するための前提である。

以降の節では、個別の拡張候補について

どの章・どの規則に関連するか

現時点で標準仕様に含めていない理由

将来導入する場合の影響範囲

を明確にし、拡張仕様の導入・廃止を判断する際の材料として 利用できる形で記録していく。

## 9.2 V モーラ拡張候補（eo/ae 系）



本節では、5.7.5～5.7.7 で扱った eo/ae 系 V モーラ例外のうち、標準仕様の外縁に位置づけられる拡張候補を整理する。

特に sieawo に代表される e + a 系の拡張は、標準仕様と拡張仕様の境界を考えるうえで重要な観測対象となる。

---

### 9.2.1 標準仕様側の eo/ae 例外 (siteori / osaete)

まず、5.7.5 および 5.7.6 で定義された eo/ae 系の標準的な V モーラ例外を整理する。

siteori (5.7.5, eo 系)

tokens: CV:si, CV:te, V:o, CV:ri

moras: CV:si, CV:te, V:o, CV:ri

result: SiTeORi

te + o の並びにおいて、後続の V モーラ o を立てる規則。

V 系回帰テスト表 2.5 および 7.2 の代表語として標準仕様に含まれる。

osaete (5.7.6, ae 系)

tokens: V:o, CV:sa, V:e, CV:te

moras: V:o, CV:sa, V:e, CV:te

result: OSaETe

語頭 o と中間 e の両方を立てる規則。

V 系回帰テスト表 2.6 および 7.2 の代表語として標準仕様に含まれる。

これらは、標準仕様の中核として扱われ、scope-flag も required に設定されている。一方で、sieawo はこれらを壊さずに拡張の余地を探るための外縁的な代表語として位置づけられている。

---

### 9.2.2 sieawo の位置づけ (拡張候補)

sieawo は、5.7.7 において eo/ae 拡張候補として 次のように定義されている。

tokens: CV:si, V:e, V:a, CV:wo

moras: CV:si, V:e, V:a, CV:wo

result: Sieawo

規則: si の後にある e を評価対象とし、  
後続の a は据え置く (e + a 連続に対して先行 e を立てる)。5.7.7 では、この規則を次のように位置づけている。

eo / ae 系拡張の妥当性を観測するための補助的なルール。

標準仕様の中心例ではなく、既存の siteori (eo) ・ osaete (ae) を  
壊さずに外縁を試すための観測用代表例。7.3 では、scope-flag を extension として設定し、  
V 系回帰テスト表 2.7 「sieawo 系」と対応付けたうえで、  
拡張仕様側に属する代表語として整理している。

### 9.2.3 拡張仕様としての採用条件 (案)

sieawo に対応する e + a 系拡張規則を 標準仕様に格上げするかどうかは、運用上の判断に委ねられる。

本ガイドラインでは、次のような条件を満たした場合に 採用を検討する、という方針案を記録しておく。

他の e + a 系語に対しても、sieawo と同様の  
立て方を適用したい需要が明確に存在すること。

siteori (eo) ・ osaete (ae) など、既存の eo/ae 例外との  
競合や矛盾が生じないこと。

回帰テスト表 2.x において、sieawo 以外の  
e + a 系代表語を追加することが検討されていること。

仕様書読者にとって、e + a 系拡張を  
標準仕様として受け入れやすい文脈が整っていること  
(例: 三段ログや代表語リストに十分な説明と例がある)。これらの条件を満たさない段階では、  
sieawo は引き続き  
拡張候補 (scope-flag = extension) として扱うことを推奨する。

### 9.2.4 拡張仕様のリスクとメリット

sieawo に代表される V モーラ拡張には、次のようなメリットとリスクがある。

#### メリット

eo/ae 系以外の母音連続 (e + a など) にも  
一貫した「先行母音を立てる」パターンを提供できる。

代表語テスト表 2.7 を通じて、  
拡張仕様の挙動を継続的に観測・検証できる。

## リスク

既存の標準仕様に含まれない  $v$  例外が増えることで、読者や実装者にとって仕様の複雑さが増す。

将来的に他の  $e + a$  系語を追加した際、一貫したルールに一般化しにくい場合がある。

これらの点から、本ガイドラインでは当面 `sieawo` を拡張仕様に留め、標準仕様側の `eo/ae` 例外 (`siteori`・`osaete`) とのバランスを重視する方針とする。

### 9.2.5 今後の検討メモ

$V$  モーラ拡張候補については、次のような検討事項が今後のタスクとして想定される。

`sieawo` 以外の `eo/ae/ea` 系代表語の追加候補が現れた場合、それらを 7 章・8 章に追加し、三段ログと回帰テスト表 2.x に反映する手順を 9.4 のポリシーに従って実行する。

`eo/ae` 系以外 (例: `io`, `ua` など) の  $v$  連続についても、拡張候補として扱うかどうかを 9.3 と連動して検討する。

拡張仕様を長期運用する中で、実際の利用頻度や可読性への影響を観測し、標準仕様に取り込むかどうかを判断する。これらのメモは、将来の仕様拡張時に「どの候補からどの順番で標準仕様に昇格させるか」を決める際の参考情報として用いる。

## 9.3 ローマ字判定・適用スコープの拡張候補

本節では、6 章で定義したローマ字判定および すばる式強調の適用スコープについて、現時点では採用していないが将来的に検討し得る拡張候補を整理する。

ここで挙げる項目はいずれも、標準仕様には含めず、拡張仕様または保留事項として扱う。

### 9.3.1 文字集合（半角英数以外）の拡張候補

6.1.1 では、ローマ字判定・すばる式強調の対象文字を 半角英数 (ASCII の `A~Z`, `a~z`, `0~9`) に限定した。

これは、マルチバイトのラテン文字 (アクセント付きなど) や 全角アルファベットを扱う際の文字コード問題を 本ガイドラインの範囲外とするためである。

拡張候補としては、次のような方向が考えられる。

### アクセント付きラテン文字の受理

例: é, ä, ñ など、  
内部的には e, a, n と同等に扱うかどうか。

ローマ字判定において、「見かけ上ローマ字に近いもの」として  
許容する案。

### 全角アルファベットの正規化

ユーザー入力で ABC のような全角アルファベットが  
混入した場合に、半角英数へ正規化してから  
ローマ字判定・強調対象とする案。

これらは、国際化や多言語環境での利用を想定した拡張であるが、  
文字コード変換や正規化処理を伴うため、  
現行の標準仕様では採用しない。  
将来、本ガイドラインを他言語環境へ広げる必要が生じた場合に、  
拡張仕様として再検討する。

## 9.3.2 単語境界の拡張候補（スペース以外の区切り）

6.1.2 では、「単語」を入力文字列のスペース区切り単位と定義した。

これは、上位システムがあらかじめ分かち書きを行っていることを前提とした簡潔な定義である。

拡張候補としては、次のような単語境界の解釈が考えられる。

### 句読点・カンマ・ピリオドを単語境界として扱う

例: foo,bar を foo / bar に分割する。

### ハイフン・アンダースコアを含む記号連結を 単語内に含めるかどうかの細分化

例: foo-bar を 1 語として扱うか、  
foo / bar に分割するか。

これらの拡張は、ローマ字以外の語や英単語、  
プログラミング言語の識別子を扱う際に有用である可能性があるが、  
単語定義を複雑化するため、現行の標準仕様では  
「スペース区切りのみ」を採用している。

実装側で追加のトークナイズ処理を行う場合は、  
本ガイドラインとは別のレイヤーの仕様として定義する。

## 9.3.3 ローマ字判定の厳密化・緩和

6.2 では、ヘボン式と訓令式の混在を許容しつつ、「ローマ字らしさ」をおおまかに確認するトリガー判定と位置づけた。

拡張候補としては、次の二方向の調整が考えられる。

#### 厳密化の方向

特定のローマ字体系（例：ヘボン式）への準拠を強く求めるモードを追加する。

例：si / ti を不正とし、shi / chi のみを受理する。

これは、教育用途や特定ドメイン向けに厳格なローマ字表記を求める場合に有用である。

#### 緩和の方向

より自由な英字列（例：英単語や略語）もローマ字判定の対象に含めるモードを追加する。

例：file, hash, id などを「ローマ字に近い英単語」として扱う。

これらのモード切り替えは、実装仕様側でローマ字判定モジュールの挙動を設定可能にすることで対応できるが、本ガイドラインの標準仕様では「混在許容・緩めの判定」を基本とし、厳密化・緩和は拡張仕様として扱う。

### 9.3.4 長大トークン閾値 L の拡張候補

6.4 では、半角英数連続長が 32 文字以上の単語を 長大トークン として無条件に無変換とするデフォルト閾値を定めた。

また、オプションとして  $1 \leq L \leq 200$  の範囲で 閾値 を変更できる仕様とした。

拡張候補としては、次のような運用が考えられる。

ロジック側で L を 32 より小さく設定し、短めの識別子や ID も強調対象から外すモード。

逆に、L を 64 以上に設定し、SHA-256 などの長いハッシュだけを除外するモード。

これらは、本ガイドラインの仕様としては既にオプションとして許容されているが、実運用上はどの値を採用するかを慎重に決める必要がある。

9 章では、L のデフォルト値を 32 に設定した経緯

(MD5 など一般的なチェックサムを安全側で除外するため) を記録しておき、将来デフォルト値を変更する場合の検討材料とする。

### 9.3.5 今後の拡張検討に向けたメモ

ローマ字判定および適用スコープの拡張候補は、いずれも仕様の射程と複雑さに直結するため、次の観点を踏まえて慎重に検討する必要がある。

#### 読者・利用者にとっての理解しやすさ

半角英数限定・スペース区切りという現在の制約は、仕様を読みやすく保つうえで有効に機能している。

#### 実装コストとバグリスク

文字集合や単語境界の拡張は、実装・テストの負荷およびバグリスクを増加させる。

#### 利用シーン

どの程度「日本語ローマ字以外の入力」を対象にしたいか（英語・他言語・識別子等）に応じて、拡張の優先度を判断する。

本ガイドラインでは、現時点では 6 章で定めた範囲を標準仕様とし、9.3 に記載した内容は将来の拡張検討のためのメモとして位置づける。

## 9.4 拡張仕様の導入・廃止ポリシー

本節では、9.1～9.3 で挙げた拡張・保留項目を 実際に導入・廃止する際のポリシーを定義する。

このポリシーは、拡張仕様を安易に標準仕様へ組み込んだり、逆に撤回する際に一貫性を失ったりしないための運用上の指針として機能する。

### 9.4.1 拡張仕様導入時のチェックリスト

拡張仕様（scope-flag が extension または experimental）を 標準運用で有効にする、あるいは標準仕様に昇格させる場合、少なくとも次の点を確認する。

#### 影響範囲の特定

対象となる拡張仕様が、どの章・どの節に関連するか  
（例：5.7.7 sieawo、6.2 のローマ字判定、6.4 の長大トークン閾値、など）を明確にする。

## 5 章の本文の更新

新たに標準仕様として採用する表示規則がある場合、  
5 章本文にその規則を明示的に追記する。

拡張仕様として記述していた章内の文言（例：「拡張候補」）を、  
必要に応じて標準仕様向けの表現へ修正する。

## 7 章代表語リストの更新

該当する代表語行の `scope-flag` を  
`extension` / `experimental` から `required` に更新する。

必要に応じて `category`・`notes` を調整し、  
標準仕様の代表語として読みやすい形に整える。

## 三段ログと 8 章の更新

該当する代表語の三段ログ行が存在しない場合、  
8.1 で定義したフォーマットに従って新規追加する。

既存の三段ログ行がある場合は、  
`rules-applied`・`test-table-ref`・`scope-flag` などを  
標準仕様に合わせて更新する。

## 回帰テスト表の更新

`required-regression-test-plan.md` において、  
該当するテストケースを「必須回帰」の範囲へ移す。

1.x / 2.x のどの節に属するかを明確にし、  
必要に応じて説明文（コメント）を更新する。

これらのステップを経て初めて、「拡張仕様を標準仕様として導入した」  
と見なすことができる。

### 9.4.2 拡張仕様廃止時のポリシー

一度導入した拡張仕様を廃止する場合も、標準仕様と同様に慎重な手順を取る必要がある。

#### 廃止対象の明確化

どの代表語・どの規則（例：特定の `v` 例外）が  
廃止対象かを明確にする。

影響を受ける 5～8 章の箇所をリストアップする。

#### `scope-flag` の変更

廃止する仕様に対応する代表語行の `scope-flag` を

required から extension または experimental に変更する。

完全に削除するのではなく、当面は「非推奨・廃止予定」として状態を明示することを推奨する。

## 5 章本文の修正

廃止される規則が 5 章本文に記載されている場合、その記述を削除するか、9 章への参照（拡張・保留項目）として移動する。

読者が「現在有効な標準仕様」と「過去に存在した拡張仕様」を区別できるようにする。

## 回帰テスト表の更新

該当するテストケースを「標準の必須回帰対象」から外し、拡張・任意テストとして区別する。

必要に応じて、テスト表内で extension などのラベルを明示する。

### 9.4.3 標準仕様への昇格・降格の判断基準

拡張仕様を標準仕様に昇格させる／標準仕様から拡張仕様へ降格させる判断は、次の観点を総合的に考慮して行う。

#### 利用頻度

実際の運用において、その規則・代表語がどの程度利用されているか。

利用頻度が高く、拡張仕様のままでは運用上不便が大きい場合、昇格を検討する価値がある。

#### 可読性・一貫性

その規則を標準仕様に含めることで、他の規則との一貫性が高まるか、逆に複雑さが増すか。

例：sieawo のような拡張が、siteori・osaete とのバランスを崩さないか。

#### 実装・テストコスト

拡張仕様を標準化した場合の実装・テストコストが受け入れ可能な範囲か。

既に実装・テストが十分整備されている拡張仕様は、



昇格コストが低い。

#### 仕様の安定性

拡張仕様自体がまだ変動しやすい場合  
(例: ローマ字判定のルールが頻繁に変わる)、  
標準仕様を含めるには時期尚早と判断する。

### 9.4.4 拡張仕様変更時のレビュー体制

拡張仕様の導入・廃止・昇格・降格は、標準仕様と同様にレビューを前提とする。

#### レビュー対象

- 5 章本文の変更箇所
- 7 章代表語リストの `scope-flag`・`category` の変更
- 三段ログ (8 章) の該当行
- 回帰テスト表の該当テストケース

#### レビュー観点

- 6 章の適用スコープと矛盾がないか。
- 7 章の代表語リストと三段ログが同期しているか。
- 仕様書読者にとって混乱を招く表現がないか。

これらのレビューを通じて、  
拡張仕様の扱いが場当たり的な変更にならないようにする。

### 9.4.5 本節の位置づけ

本節で定めた導入・廃止ポリシーは、拡張仕様を「実験的に試しつつも、いつでも安全に戻せる」状態で維持するための枠組みである。

これにより、本ガイドラインは標準仕様としての安定性と、拡張仕様としての柔軟性を両立させることを目指す。

今後、sieawo を含む V モーラ拡張や ローマ字判定・適用スコープの拡張を検討する際には、本節のポリシーに基づいて導入・廃止を判断する。

## 更新履歴

| 版              | 更新日        | 主な変更点概要                                                                                                                                                                                                                                        |
|----------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ver 2.1-r2     | 2026/06/15 | <ul style="list-style-type: none"><li>• sieawo を SieaWo として扱い、<br/>V-ea-extension（連母音塊維持）の代表語に設定</li><li>• seikou / keiei / seion<br/>など ei 系一般化の代表語セットを整理</li><li>• Ver 2.1 系 UI デモおよび Part-004 検討の基準版</li></ul>                              |
| Ver 2.2-draft1 | 2026/06/16 | <ul style="list-style-type: none"><li>• V-eo / V-ae / V-ea を標準特例として揃え、<br/>sieawo 標準表示を SieAWo に更新</li><li>• 7.2 / 7.3 に V 代表語と拡張履歴を整理し、<br/>scope-flag / phase を導入</li><li>• 8 章表示例と回帰テスト Block NQ / Block V を<br/>三段ログ仕様と合わせてリンク付け</li></ul> |
| Ver 2.3        | 2026/06/18 | <ul style="list-style-type: none"><li>• 代表語テーブルと三段ログ／表示例を再編</li><li>• required 回帰テスト表と対応付け</li><li>• V モーラ／N モーラ代表語を系統的に整理</li><li>• V 系標準代表セットを昇格・明示</li><li>• V 拡張語を履歴と拡張候補スロットとして整理</li></ul>                                               |